

Schema-Agnostic Rule Extraction from Industrial Standard Operating Procedures: A Multi-Agent LLM Pipeline and a Content-First Evaluation

Dleffle

August 7, 2026

Abstract

Extracting operating rules from industrial Standard Operating Procedures (SOPs) is difficult because the information is scattered across prose, data tables, and access matrices, and because every facility documents its rules in its own vocabulary. The prevailing way to obtain high accuracy is to write an extraction prompt around one corpus’s schema (its field names, its rule taxonomy, its identifier convention) but a prompt built that way cannot be applied to a facility it was not written for without being rewritten by hand. Evaluation compounds the problem: exact-match metrics penalise an extraction for using a different name for the same quantity, confusing nomenclature with correctness.

This thesis asks what rule extraction costs when nothing about the target schema is supplied. We present a multi-agent pipeline, implemented in LangGraph, that assumes only that its input is an operating document. It segments each document and numbers every content line; parallel *scout* agents read the numbered chunks and emit records whose field names are copied from the document’s own labels; a single *inducer* call reconciles those labels into one corpus-level schema without ever seeing a document; deterministic assembly emits one row per rule; and parallel *audit* agents re-read each record against the lines it cites and drop what is not stated there. No stage is given a target schema, a rule taxonomy, or a worked example.

Extraction is scored by $F1_{\text{content}}$, a content-first metric that collapses each ground-truth row and each predicted record into a text blob, blends SBERT cosine similarity with numeric overlap, and pairs the two sides by Hungarian optimal assignment, so that identifier and field-name conventions never enter the score. Because such a metric can flatter a system without measuring it, its validity is established empirically rather than asserted: predictions scored against a foreign corpus’s ground truth collapse to 0.02–0.11, corrupting every numeric value costs 0.30 F1, and the reported operating point is a strict local maximum on none of the four corpora.

The pipeline is evaluated on one development corpus and three previously unseen external corpora (biogas digestion, cleanroom access control, and reverse-osmosis desalination) with five repeats each and no per-corpus configuration of any kind. It reaches $F1_{\text{content}} = 0.756 \pm 0.077$ on the development corpus and 0.806 ± 0.022 , 0.747 ± 0.000 , and 0.820 ± 0.045 on the three external corpora. Because the hosted endpoint is non-deterministic for long generations, every figure is reported as a mean and standard deviation over repeats rather than as a single run.

Set against this, a grid search over ten prompting paradigms and two models, in which the prompt is given the development corpus’s schema, taxonomy, identifier convention, and worked examples, reaches 0.946 ± 0.017 on that corpus, and cannot be run on the other three at all without being rewritten. Notably, the two systems recover the ground truth almost equally well (recall 0.884 against 0.907); the gap is overwhelmingly precision, because the two disagree about what constitutes one rule and any surplus record is counted as an error whether or not it is correct. The gap is therefore best read as the measured price of requiring no per-domain configuration, and the schema-specified condition as an instance of the corpus-specific adaptation this work seeks to avoid.

CONTENTS

1	Introduction	5
1.1	Context and Motivation	5
1.2	Research Gaps	6
1.3	Contributions and Objectives	7
2	Theoretical Background	7
2.1	Large Language Models	8
2.2	Knowledge Graphs and Graph Databases	9
2.3	Multi-Agent Systems and LangGraph	10
2.4	Industrial Digital Twins and Standard Operating Procedures	11
3	Related Work	11
3.1	Document-Level Information Extraction and Evaluation	11
3.2	LLM-Based Knowledge Graph Construction	12
3.3	Knowledge Graphs for Industrial Digital Twins	14
4	System Overview	14
5	Unstructured Data Parsing Strategy	15
5.1	Overview of the Document Parser	15
5.2	Vision-Aware Markdown Extraction	15
5.3	Normalisation and Hallucination Scrubbing	16
5.3.1	Ligature and Encoding Correction	16
5.3.2	Table Cell Deduplication	16
5.4	Context-Aware Chunking Strategy	16
6	Methodology	17
6.1	Research Design and Experimental Overview	17
6.2	Corpora: One Development Corpus and Three External Test Corpora	18
6.2.1	Development Corpus	18

6.2.2	Facility Structure	18
6.2.3	SOP Document Corpus and Structural Heterogeneity	19
6.2.4	External Test Corpora	20
6.2.5	Ground-Truth Rule Schema	20
6.3	Experimental Conditions	21
6.3.1	Inference Endpoint	21
6.3.2	Language Models	21
6.3.3	Temperature, Seed, and the Limits of Reproducibility	22
6.3.4	Chunking and the Output Budget	22
6.4	Evaluation Metric Design: Motivation and Framework	23
6.4.1	Limitations of Standard Exact-Match Metrics	23
6.4.2	Design Principles for $F1_{\text{content}}$	23
6.5	The Metric: Content Agreement over Optimally Assigned Pairs	24
6.5.1	The Procedure End to End	24
6.5.2	A Worked Example	25
6.5.3	Why There Is No Field-by-Field Comparison	26
6.5.4	Blob Construction (Steps 1–2)	28
6.5.5	Agreement Between Two Blobs	29
6.5.6	Optimal Assignment and $F1_{\text{content}}$	29
6.5.7	Free Parameters	30
6.5.8	The Precision Ceiling Imposed by Granularity	30
6.5.9	Identifier Reuse as a Diagnostic, Not a Score	31
6.6	Establishing That the Metric Measures Anything	31
6.6.1	The Null Case	31
6.6.2	Perturbation Tests	32
6.6.3	Where the Operating Point Sits	32
6.7	Reviewability: The Per-Record Audit Trail	33
7	Schema-Specified Extraction: The Prompting Grid	34
7.1	What the Prompt Is Given	34
7.2	Models and Experimental Parameters	35
7.3	Prompting Paradigms and Complexity Levels	35
7.4	Ensemble Voting and Content Agreement	36
7.5	System Resilience and Output Parsing	36
8	Multi-Agent Extraction Pipeline Architecture	37
8.1	Stage 1: Segmentation and Line Numbering	39
8.2	Stage 2: Scout Agents	39
8.3	Stage 3: Schema Induction	40
8.4	Stage 4: Deterministic Assembly	40

8.5	Stage 5: Grounding Audit	41
8.6	Model Assignment	41
8.7	Error Resilience	41
9	Results	42
9.1	Extraction Across Four Corpora	42
9.2	Field-Level Verification Against the Source Documents	43
9.3	The Cost of Requiring No Schema	44
10	Limitations	45
10.1	Granularity Disagreement Bounds Every Reported Figure	46
10.2	The Metric Does Not Read Word Order or Attribute Binding	46
10.3	The Metric Has a Non-Zero Floor	46
10.4	One Corpus Is Threshold-Unstable	46
10.5	Ground-Truth Incompleteness	47
10.6	Results Are Not Bit-Reproducible	47
10.7	Corpus Scale and Composition	47
10.8	Five Repeats Bound the Statistical Claims	48
10.9	Single Model, Single Endpoint	48
10.10	No Downstream Validation	48
A	Complete Evaluation Results	48

1. INTRODUCTION

1.1 Context and Motivation

A radical paradigm shift has been undergone by the field of Knowledge Extraction (KE), evolving from rigid, domain-specific architectures toward fluid, generative paradigms. Historically, the identification of predefined entities and relations within unstructured text was heavily reliant on fixed ontologies, which created significant bottlenecks when processing the vast volume and variety of modern digital data. With the widespread adoption of transformer architectures, manual heuristics have been progressively replaced by deep contextual representations. Consequently, a robust technical foundation has been provided for the conversion of raw digital signals and textual documentation into structured Knowledge Graphs (KGs).

Within modern industrial manufacturing facilities, KGs serve as the critical cognitive backbone for intelligent Digital Twins (DTs). However, high-quality KG construction represents a critical bottleneck for knowledge-driven digital twins, as traditional approaches such as manual ontology engineering or centralized natural language processing (NLP) pipelines fail to scale while maintaining semantic richness. To construct these KGs, operational knowledge must be systematically extracted from heterogeneous sources, primarily Standard Operating Procedures (SOPs). These documents encode essential parameters, including alarm thresholds, maintenance rules, and access policies. Theoretically, all operational logic required to synchronize a digital twin with its physical counterpart is encoded within these texts. In practice, however, the automated extraction of this knowledge is severely obstructed by multiple challenges.

First, a high degree of structural heterogeneity is exhibited by SOPs within a single facility. Critical information is frequently dispersed across fundamentally different layouts; for instance, modern industrial facilities generate knowledge simultaneously in natural language documents, numerical sensor streams, and manufacturer datasheets. The effective processing of these distinct formats cannot be consistently achieved by a single, monolithic extraction prompt, as the information is spatially and semantically structured in fundamentally divergent ways.

Second, the evaluation of automated extraction in industrial contexts is highly complex. A phenomenon defined as “semantic aliasing” is frequently observed, where identical operational constraints are mapped to disparate identifiers. In ground-truth annotations, rule identifiers, such as RULE-ST01-01 or MAINT-01, are often established as annotator conventions that rarely appear verbatim in the source text. When evaluation relies on exact-match F1 over identifiers, taxonomic nomenclature is confused with true semantic accuracy. Consequently, an “evaluation blind spot” is created, which risks penalizing operationally valid extractions while simultaneously failing to detect hallucinated but syntactically correct numeric thresholds.

Large Language Models (LLMs) are considered highly attractive for this extraction task, given their capacity to interpret complex natural-language instructions and adapt to new extraction patterns. Nonetheless, significant reliability issues persist; LLMs are prone to hallucinating numeric values absent from the source text and possess a “black-box” nature that obscures their decision-making processes. To mitigate the limitations of monolithic generative models, research is increasingly being directed toward collaborative multi-agent architectures. In these advanced frameworks, a collaborative multi-agent paradigm is proposed where specialized software agents, such as document parsers, entity extractors, and ontology alignment

agents, are orchestrated to extract, validate, and integrate knowledge across heterogeneous formats. Recent literature highlights several successful implementations of this paradigm; for example, the KARMA framework deploys nine collaborative LLM agents for entity discovery and conflict resolution, while Tag2Graph is explicitly designed to mine KGs from industrial time-series data and tag definitions.

Bridging the gap between the generative flexibility of LLMs and the deterministic rigor required by industrial SOPs remains a primary experimental frontier. This thesis investigates the design, implementation and evaluation of a collaborative multi-agent system for that task, on four corpora from unrelated industrial domains: a nine-station packaging facility used for development, and three previously unseen corpora covering biogas digestion, cleanroom access control and reverse-osmosis desalination.

1.2 Research Gaps

Although multi-agent systems have significantly expanded the capacity for generative information extraction, systematic evaluation of these pipelines is constrained by a lack of specialised benchmarks. The review in Section 3 indicates an absence of systematic architectures for agent specialisation in extraction from industrial documentation, and a lack of comparative studies on agent collaboration strategies. Most consequentially for this work, the reported accuracy of such pipelines is almost always obtained on the same corpus their prompts were written around, so it is not known what they achieve on documentation they were not designed for.

From an evaluation perspective, existing work fails to simultaneously address the unique demands of industrial SOP extraction. Specifically, three critical requirements remain unfulfilled by the current state of the art:

- **Extraction that is not written around one corpus.** Reported accuracy on rule extraction is typically obtained by encoding the evaluation corpus’s own schema, rule taxonomy and identifier convention into the prompt. Such a system is not transferable: applying it to a second facility requires a human to read that facility’s documents and rewrite the prompt. What a pipeline achieves when nothing about the target schema is supplied is rarely measured, and is the question this thesis addresses.
- **Evaluation across corpora rather than within one.** A single annotated corpus cannot distinguish a general extraction capability from a system tuned to the documents it is scored on. Establishing the distinction requires previously unseen corpora from unrelated domains, scored without any per-corpus adjustment, and repeated often enough to separate a real difference from endpoint noise.
- **A content-first metric whose own validity is demonstrated.** Scoring operational correctness independently of annotator-defined naming requires semantic matching (Hungarian assignment over sentence-embedding similarity), but such a metric can assign a flattering score without discriminating anything, because embeddings place any two documents of the same genre at a non-trivial similarity. A metric introduced for this purpose must therefore be accompanied by evidence that it collapses when the extraction is wrong, and by a statement of what it does not measure.

This thesis responds to these gaps with a pipeline that receives no schema, an evaluation spanning four

corpora from unrelated industrial domains, and a metric whose discriminating power and blind spots are both measured and reported.

1.3 Contributions and Objectives

To address these limitations, this work makes four primary contributions:

- **Schema-Agnostic Multi-Agent Pipeline:** A LangGraph extraction pipeline whose only assumption about its input is that the input is an operating document. Field names are copied from each document’s own labels by parallel scout agents, reconciled into a corpus-level schema by a single inducer call that never sees a document, assembled deterministically, and checked by parallel audit agents against the numbered lines each record cites. No stage receives a target schema, a rule taxonomy, or a worked example, so the same pipeline invocation runs unchanged on a facility it has never seen.
- **Content-First Evaluation Metric:** $F1_{\text{content}}$, which collapses each ground-truth row and predicted record into a text blob, blends SBERT cosine similarity [20] with numeric overlap, and pairs the two sides by Hungarian optimal assignment. Neither side’s field names or identifier conventions enter the score, which is what makes a comparison possible at all when the extractor discovers its own vocabulary.
- **Empirical Validation of the Metric Itself:** A metric built on sentence embeddings can post a respectable score without discriminating anything, so its validity is measured rather than argued. Predictions scored against a foreign corpus’s ground truth fall to 0.02–0.11; corrupting every numeric value costs 0.30 F1; and the reported threshold is a strict local maximum on none of the four corpora, so it cannot have been chosen to flatter the system. The same study reports, with equal prominence, what the metric does *not* measure: word order within a record, which value is bound to which record, and the correctness of the discovered category label.
- **Cross-Corpus Evaluation and the Cost of Schema-Agnosticism:** Evaluation on one development corpus and three previously unseen external corpora (biogas digestion, cleanroom access control, reverse-osmosis desalination), five repeats each, with no per-corpus configuration. Against this, a grid search over ten prompting paradigms and two models whose prompt is given the development corpus’s schema, taxonomy, identifier convention and worked examples scores markedly higher on that corpus and is inapplicable to the others without being rewritten. Because both systems recover the ground truth to within 0.023 recall and differ chiefly in how finely they itemise it, the gap quantifies what requiring no per-domain configuration costs, and identifies the schema-specified condition as corpus-specific adaptation rather than a fair baseline.

2. THEORETICAL BACKGROUND

In this chapter, the foundational concepts and computational tools upon which the remainder of the thesis is built are introduced. The theoretical framework encompasses several interconnected domains, including

deep learning, natural language processing (NLP), agentic artificial intelligence, and Knowledge Graphs (KGs). The primary objective is to provide sufficient theoretical context to elucidate the design decisions detailed in subsequent chapters, ensuring comprehensibility without assuming prior expertise in any specific technological paradigm.

2.1 Large Language Models

Large Language Models (LLMs) are characterized as advanced deep learning models that have been extensively pre-trained on massive textual corpora to understand context and generate human-like responses. The foundational architecture of nearly all modern LLMs is the transformer, which utilizes a self-attention mechanism to process sequences more efficiently and accurately by relating any token in an input to any other token. Various architectures have been developed within this paradigm, including encoder-only models (e.g., BERT) for deep contextual understanding, decoder-only models (e.g., GPT) for text generation, and encoder-decoder models (e.g., T5) for highly flexible text-to-text transformations.

Two properties of particular relevance to this thesis are zero-shot and few-shot in-context learning [14]. In zero-shot settings, a model follows task instructions expressed entirely in natural language without any worked examples, enabling novel extraction tasks with no labeled data. Few-shot in-context learning extends this by embedding a small number of worked examples directly in the prompt, guiding the model toward the desired output format or extraction pattern without task-specific fine-tuning or weight updates. Both capabilities render LLMs attractive for knowledge extraction in industrial settings where annotated training corpora are scarce, a direct constraint here, where the four evaluation corpora provide between 26 and 86 annotated rules each, 196 in total. Chain-of-Thought prompting [22], which requires the model to produce explicit intermediate reasoning steps before its final answer, is a complementary technique that improves extraction reliability by making intermediate decisions inspectable. Together, zero-shot prompting, few-shot in-context learning, and Chain-of-Thought reasoning represent application-level enhancements that improve LLM utility without modifying model weights [2], the paradigm this thesis operates within entirely, as no fine-tuning or weight adaptation of any model is performed.

Despite these capabilities, LLMs exhibit significant limitations that are especially consequential in industrial contexts. The most critical is the generation of *hallucinations* [17]. Two distinct hallucination modes are distinguished in the literature: *intrinsic hallucination*, where the output directly contradicts information present in the source document, and *extrinsic hallucination*, where the output introduces content that cannot be verified against the source at all. Both are dangerous for industrial SOP extraction, but extrinsic hallucination is particularly insidious: a numeric threshold value invented by the model and absent from the source text will not be flagged by any input-grounding check, yet could cause a safety alarm to fire at an incorrect value or fail to trigger when it should. A second limitation is difficulty with precise numerical reasoning: numeric values embedded in multi-column threshold tables with engineering units are susceptible to transcription errors in which adjacent row values are transposed or incorrect unit suffixes are appended. Third, the “black-box” nature of LLM inference makes it difficult to trace why a specific prediction was produced, complicating systematic debugging of extraction errors. The mitigation of these limitations (hallucination control, numeric grounding, and reasoning transparency) are primary design concerns addressed

by the multi-agent pipeline proposed in Section 8.

Models are served through an OpenAI-compatible endpoint (Section 6.3.1); nothing in the pipeline depends on that endpoint being remote rather than local, and the same code runs against either. The choice does carry one methodological consequence that shapes every reported figure: a shared endpoint batches requests with other traffic, so long generations are not reproducible under a fixed seed, and results must be reported as a mean with a standard deviation over repeats rather than as a single run. Section 6.3.3 develops this point, which is why no result in this thesis is presented as a single number.

2.2 Knowledge Graphs and Graph Databases

A Knowledge Graph (KG) is defined as a structured representation of knowledge where entities and their relations are linked within a graph. The core structural elements include nodes, which represent entities or concepts, and edges, which denote the relationships between these entities. Properties or attributes are frequently added to provide supplementary factual data, such as dates, locations, or numerical values. KGs provide a semantic framework that supports complex data queries and reasoning, successfully narrowing queries to small data subsets and maintaining high performance regardless of dataset growth. A central design philosophy is to connect *concepts rather than strings* [8], enabling semantic relationship traversal rather than keyword lookup; practical applications of this paradigm include search optimisation, question-answering systems, and recommendation engines. The overall quality and utility of a KG are strictly dependent on the quality of the data fed into it, a principle that directly motivates the content-first evaluation metric introduced in this thesis.

Constructing a KG from unstructured text relies on three core NLP tasks [3]. *Entity extraction* (Named Entity Recognition, NER) identifies key entities in a document (persons, organizations, locations, sensor identifiers, operational parameters) that become the nodes of the graph. *Relation extraction* then infers typed relationships between pairs of identified entities from their textual context, producing the labelled edges that encode operational constraints and causal dependencies. A third task, *entity resolution*, determines whether multiple text mentions refer to the same real-world entity, consolidating duplicate candidate nodes before they are committed to the graph. Whereas early methods relied on hand-crafted syntactic rules and sequence-labelling models such as Conditional Random Fields, modern approaches adapt pre-trained language models to each extraction task through supervised fine-tuning or, as in this thesis, zero- and few-shot prompting. Two broad construction strategies are distinguished in practice [8]: an *NLP-only* approach that extracts subject–verb–object triples directly from text (maximally flexible but susceptible to entity disambiguation errors when the same surface form refers to different entities), and an *NLP-lite* approach that identifies named entities and resolves them against a structured knowledge base such as Wikidata to standardise relationship types (built-in disambiguation, but constrained to pre-specified relation types).

The organization of KG content is heavily influenced by Description Logics, utilizing a bifurcated structure:

- The **TBox** (terminological box) establishes the domain ontology, defining the schema, entity types (e.g., Component, Process, Parameter, Sensor), relationship types (e.g., *part_of*, *monitors*, *triggers*), and property constraints.

- The **ABox** (assertional box) contains the concrete instance data (the specific sensors, stations and extracted rules of one facility) populated against the schema the TBox defines.

Graph databases such as Neo4j implement this model natively and expose it through query languages such as Cypher, supporting the multi-hop traversal that downstream analytics require. That downstream stage is the motivation for this work rather than part of it: this thesis addresses the step that must succeed first (extracting the rules) and evaluates that step against annotated ground truth. Whether the extracted rules are operationally sufficient to drive graph-based analytics is a separate question, and Section 10.10 states plainly that it is not answered here.

A growing body of research explores the interaction between LLMs and KGs more broadly [19]. Three principal integration paradigms are identified: *LLM-augmented KGs*, where LLMs serve as the primary extraction engine to populate a graph from unstructured text; *KG-augmented LLMs*, where structured graph knowledge is injected into the LLM context to reduce hallucination and improve factual grounding; and *synergized frameworks*, where both modalities reinforce each other iteratively. This thesis sits within the first paradigm (LLMs as the extraction engine that would populate such a graph) and stops at the extraction boundary, where the output is a set of typed rule records with per-record provenance rather than a populated graph.

2.3 Multi-Agent Systems and LangGraph

To overcome the limitations of centralized NLP pipelines, which frequently fail to scale while maintaining semantic richness, a collaborative multi-agent paradigm is increasingly adopted. A multi-agent system (MAS) is defined as a software architecture comprising multiple autonomous, specialized components, termed agents, that interact to achieve complex objectives. Various collaboration strategies can be supported by these architectures, including cooperative, competitive, and hierarchical frameworks.

The primary advantage of a multi-agent approach is agent specialization. Within an extraction pipeline, dedicated agents can be deployed for specific tasks, such as document parsing, domain-specific named entity recognition (NER), relationship extraction, ontology alignment, and validation. Recent literature highlights the efficacy of this approach in industrial contexts. The KARMA framework [6] employs nine collaborative LLM agents to mine a multi-source industrial dataset, extracting 38,230 entities at 83.1% LLM-verified correctness while achieving an 18.6% reduction in inter-agent entity conflicts through structured majority voting. Analogous frameworks such as Tag2Graph apply a collaborative paradigm to mine KGs directly from industrial time-series data and tag definitions. Multi-agent architectures have also demonstrated superior performance on standard NER and relation extraction benchmarks relative to single-agent baselines [10]. To ensure semantic consistency more broadly, conflict resolution mechanisms frequently utilize majority voting protocols with configurable confidence thresholds to assign composite reliability scores to extracted triples.

The extraction pipeline presented in this thesis is implemented using the LangGraph framework. LangGraph facilitates the organization of agents as a typed directed acyclic graph (DAG) with a shared state dictionary. Two features are load-bearing in this implementation. Parallel fan-out lets one agent role be instantiated once per document chunk and run concurrently, which is how both the extraction and the audit

stages work. Sum-reducer state fields let those concurrent instances accumulate into a shared list without overwriting one another, which is what makes the fan-out safe: each scout appends its records rather than replacing the state.

2.4 Industrial Digital Twins and Standard Operating Procedures

A Digital Twin is conceptualized as a highly synchronized software model of a physical system, fundamentally requiring sophisticated knowledge graphs to integrate diverse static and dynamic data sources. The architecture of an agentic Digital Twin typically integrates a static model, represented by the knowledge graph defining the industrial system structure, with a dynamic model processing simulated IoT data and operational states. KGs serve as the cognitive backbone of these twins, enabling bidirectional synchronization, incremental updates, and complex what-if reasoning. Sahlab et al. [9] demonstrate that KG integration delivers four concrete capabilities for industrial Digital Twins: internal cross-document linking that connects entities across heterogeneous data sources, autonomous knowledge completion that infers missing relationships from existing graph structure, automated error detection in sensor readings by reasoning over expected operational states, and semantic querying of the facility model using ontology-aware graph traversal. Recently, the concept of the “Digital Twin Agent” has emerged, incorporating autonomous decision-making and cross-system collaboration to accelerate modeling and resilience in manufacturing environments.

Within industrial facilities, Standard Operating Procedures (SOPs) function as the primary governance layer, encoding the operational rules that dictate Digital Twin behavior. These natural language documents specify critical parameters, including continuous numerical measurement thresholds, maintenance rules, and zone access policies. However, the automated extraction of this logic is complicated by the extreme structural heterogeneity of SOPs, which frequently combine narrative prose, complex alarm threshold tables, and personnel access matrices. The translation of these heterogeneous, unstructured documents into verified, machine-readable rule records (the form a graph would subsequently be populated from) is the central methodological challenge addressed by the multi-agent system developed in this thesis.

3. RELATED WORK

This chapter reviews existing work in the three areas most relevant to this thesis: document-level information extraction and evaluation, Knowledge Graph construction from text, and Knowledge Graphs for industrial Digital Twins.

3.1 Document-Level Information Extraction and Evaluation

The most widely used benchmarks for document-level relation extraction are DocRED [23] and DocILE [21]. DocRED contains approximately 100,000 relational facts extracted from Wikipedia articles, with cross-sentence dependencies that require multi-paragraph reasoning. While DocRED pioneered large-scale joint entity and relation extraction datasets, it primarily targets encyclopedic relations without engineering analogs, lacking the logical depth required for industrial constraints. Conversely, DocILE provides the largest public benchmark for Key Information Localization and Extraction (KILE) and Line Item Recognition (LIR)

across over 1,000 unique industrial layouts. Both have been important for advancing general-purpose information extraction systems, but neither addresses the characteristics specific to industrial SOP corpora: numeric sensor thresholds with engineering units, conditional operational logic distributed across heterogeneous document formats (prose, tables, matrices), and a need to evaluate extraction quality independently of how identifiers are named. Specifically, DocILE focuses on static business entities (e.g., tax totals, addresses) rather than the conditional operational rules and numeric sensor thresholds required for industrial SOP automation.

In the domain of business rules, BREX [1] represents the state-of-the-art for large-scale rule-flow modelling with conditional logic across 30+ domains. However, BREX ignores tabular structures and numeric engineering units. Furthermore, it relies on identifier-based F1 evaluation, which assumes that rule names in the source text match the ground-truth annotation convention. For industrial SOPs, rule identifiers are typically annotator-assigned rather than present in the source text, so identifier-based F1 rewards naming convention alignment as much as extraction quality. This is the core motivation for the $F1_{\text{content}}$ metric defined in Section 6.5, and for reporting identifier agreement only as a diagnostic of the annotation convention (Section 6.5.9) rather than as a score. For regulatory domains, De Jure [4] provides a robust framework for deontic logic (obligations and permissions), but it does not account for the physical threshold semantics essential for SCADA-level integration.

SOP-Bench [12] specifically targets industrial procedures and evaluates whether an agent can correctly *execute* a procedure step-by-step. This is a complementary task to the one addressed here: SOP-Bench measures task completion via action sequences, whereas this thesis addresses the prior data-fidelity step of extracting and structuring the operational rules that an execution agent would need to follow. Table 1 positions the evaluation setting used in this thesis relative to these benchmarks. The relevant contrast is not corpus size (the four corpora used here are small) but which properties of industrial rule extraction a benchmark exercises at all.

3.2 LLM-Based Knowledge Graph Construction

The use of LLMs to extract graph-structured knowledge from unstructured text has received significant attention, representing a fundamental paradigm shift from early rule-based systems and probabilistic sequence labelling toward generative extraction enabled by deep bidirectional transformers and attention mechanisms. Several recent surveys identify multiple interaction patterns between LLMs and KGs, including LLMs used to populate KGs, KGs used to ground and verify LLM outputs, and joint architectures. This thesis falls in the first category. It stops at the extraction boundary: the output is a set of rule records carrying per-record provenance, which is the input a graph-population stage would consume, and the evaluation therefore scores those records against annotated ground truth rather than scoring a graph.

Traditional KG construction systems rely on fine-tuned Named Entity Recognition (NER) and Relation Extraction (RE) models trained on labelled corpora. These approaches require substantial annotated data, which none of the corpora used here provides: their ground truths hold between 26 and 86 rules, far too few to fine-tune a dedicated extraction model. The constraint is not incidental to this study but typical of the setting: a facility documenting its own procedures does not also produce thousands of labelled extractions.

Table 1: Positioning of this thesis’s evaluation setting relative to related benchmarks. $\checkmark$ = fully addressed; $\circ$ = partially addressed; $\times$ = not addressed. *Schema-free* asks whether a system is evaluated without being given the target schema; no prior benchmark tests this, which is the gap this thesis addresses.

Benchmark	Large-scale	Primary Focus	Numeric Thresholds	Struct. Heterog.	Content-first Metric	Schema-free
DocRED [23]	$\checkmark$	Entity Relations	$\times$	$\times$	$\times$	$\times$
BREX [1]	$\checkmark$	Business Rules	$\times$	$\times$	$\times$	$\times$
DocILE [21]	$\checkmark$	Key-Value (KILE)	$\times$	$\checkmark$	$\times$	$\times$
De Jure [4]	$\checkmark$	Legal Compliance	$\times$	$\circ$	$\times$	$\times$
SOP-Bench [12]	$\times$	Agentic Execution	$\times$	$\checkmark$	$\times$	$\times$
This thesis	$\times$	Rule Extraction	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$

The prompt-engineering approach used here avoids this requirement entirely, leveraging emergent reasoning capabilities to convert raw digital signals into structured relational assets. This allows large language models to function as zero-shot learners or reasoning agents via Chain-of-Thought and ReAct paradigms, even in low-resource scenarios. The trade-off is that prompt-based LLMs are more sensitive to the exact phrasing of instructions than a trained model would be, which is part of the reason why the grid search over prompting paradigms (Section 7) was necessary. Furthermore, applying knowledge extraction to industrial SOPs remains challenging because manufacturing documents often embed critical data in non-narrative formats. To overcome the structural heterogeneity of these documents, recent research has pivoted towards multi-agent architectures that use specialized agents to coordinate extraction across different formats. For example, TabAgent [13] proposes a multi-agent framework specifically for unstructured table extraction, while SOPRAG [11] utilizes multi-view graph experts to navigate complex SOP hierarchies. Other recent frameworks, such as SPIRES [15], employ recursive extraction methods that enforce strict schema adherence.

Hallucination control is a known challenge in LLM-based extraction that has received relatively little attention in the literature for numeric fields specifically. Populating knowledge graphs from unstructured technical text remains highly error-prone due to "hallucinations" in natural language generation. In general NLP extraction tasks, a hallucinated entity name is often recoverable from context; in industrial settings, a hallucinated temperature threshold could cause a safety alarm to fire at the wrong value or not at all. The grounding discipline used in this thesis (Section 8.5) addresses the most dangerous hallucination patterns for this task: inventing numeric threshold values, appending units not present in the source, and borrowing values from neighbouring rows, columns, or sentences. Rather than relying on a prompt instruction alone,

every extracted record cites the numbered source line it was read from, and a separate audit agent re-reads the record against that line. The grounding instruction carried by every extraction prompt is deliberately compact: each numeric value and unit must be copied verbatim from the source text, and a field must be left empty rather than guessed. Compactness is itself the design choice: an instruction elaborate enough to enumerate the shapes a constraint may take teaches a model the phrasing of the corpus it was written for, which is the failure mode this pipeline exists to avoid.

3.3 Knowledge Graphs for Industrial Digital Twins

The integration of Knowledge Graphs with industrial Digital Twins has been studied in the context of fault detection, predictive maintenance, and process monitoring. Knowledge Graphs serve as the essential semantic backbone for Intelligent Digital Twins by structurally linking scattered technical data. A recurring pattern in this literature is the use of an ontology-based KG to link sensor readings to the operational rules that govern them, enabling graph-based queries such as “which rule covers this anomaly event?”.

A common limitation in this body of work is that the KG content (the operational rules, threshold values, and causal relationships) is populated manually by domain experts or from structured databases. Automating this step from unstructured SOP documents, which is the contribution of this thesis, has not been directly addressed. While recent literature demonstrates that agentic workflows are superior for automating SOP execution, a systematic evaluation of these extraction pipelines is hindered by a lack of specialized benchmarks. Current evaluation infrastructures, such as KGpipe [7] or Plumber [16], offer modular extraction pipelines but notably lack the manufacturing-specific ground truth necessary to validate SCADA-level constraints.

4. SYSTEM OVERVIEW

Before describing each component in detail, this section gives a high-level picture of the full pipeline so that the later sections are easier to follow.

The system takes a directory of operating documents as input and produces one row per extracted rule as output, together with the provenance of every row: the document and the numbered line each record was read from. The processing is organised in three layers, each described in a dedicated section:

Layer 1: Document Parsing (Section 5) Source PDFs are converted into clean plain-text files using IBM’s `docling` library for layout-aware extraction, followed by normalisation (removing OCR artefacts, fixing encoding errors), so that table structure and heading hierarchy survive into the text the extraction layer reads.

Layer 2a: Schema-Specified Grid Search (Section 7) A grid over ten prompting paradigms and two models in which the prompt is given the development corpus’s schema, rule taxonomy, identifier convention and worked examples. This establishes what is achievable when the target schema is known, and, because such a prompt cannot be applied to any other corpus without being rewritten by hand, it also delimits precisely what that accuracy is contingent upon.

Layer 2b: Multi-Agent Extraction Pipeline (Section 8) A LangGraph pipeline that receives no schema.

Documents are segmented and every content line numbered; parallel scout agents emit records whose field names are copied from each document’s own labels; one inducer call reconciles those labels into a corpus-level schema without seeing any document; assembly is deterministic; and parallel audit agents re-read each record against the lines it cites.

Layer 3: Evaluation (Section 6) Extracted rules are scored against each corpus’s ground truth by $F1_{\text{content}}$, a content-first metric using Hungarian assignment over blended semantic and numeric agreement. The same section establishes the metric’s discriminating power empirically and states what it does not measure.

The layers are separable by design. Layer 2b never reads a ground truth, so it cannot be tuned to one; Layer 3 never reads a document, so it cannot exploit knowledge of how a rule was extracted. Because every agent role is served by a hosted endpoint whose long generations are not reproducible under a fixed seed (Section 6.3.3), each configuration is run five times and reported as a mean with its standard deviation, rather than as a single figure that could not be distinguished from endpoint noise.

5. UNSTRUCTURED DATA PARSING STRATEGY

5.1 Overview of the Document Parser

Source documents arrive as PDFs, each encoding operational rules in a structurally distinct format: narrative prose, threshold tables, mixed maintenance text, role-zone access matrices. Before any LLM-based rule extraction can occur, these files must be converted into clean, semantically coherent plain text. Standard left-to-right PDF parsers fail this requirement: they flatten table structures into whitespace-separated tokens and discard hierarchical headings, making the output semantically incoherent for downstream extraction agents.

The Document Parser is the first agent in the pipeline. It accepts a raw SOP PDF as input and produces a sanitised plain-text file ready for the LLM extractors. It implements three sequential stages: vision-aware Markdown conversion, deterministic artifact sanitisation, and context-aware chunking.

5.2 Vision-Aware Markdown Extraction

To preserve spatial and semantic layout, the Document Parser uses IBM’s `docling` library, which applies a vision-language model to identify bounding boxes around text blocks, tables, and headers, exporting the reconstructed layout directly into Markdown. This matters because grid structure *is* content in these documents: a threshold table’s column position distinguishes a critical bound from a warning bound, and a permission matrix’s row and column labels are the only thing identifying which role may enter which zone. A text-only parser flattens both into an undifferentiated token stream, and no downstream stage can recover what was lost. For plain-text inputs (`.txt`), the parser bypasses vision extraction and reads the raw text directly, then applies the same normalisation and chunking pipeline.

5.3 Normalisation and Hallucination Scrubbing

Vision-based extraction preserves layout but introduces OCR artifacts and vision-model hallucinations. The parser executes a strict normalisation protocol (`_normalise`) to sanitise the raw Markdown before it enters the LLM context window.

5.3.1 Ligature and Encoding Correction

The source files contain directional and trend symbols that are corrupted by PDF font encoding: standalone `f i` tokens are deterministically remapped to a right-arrow ($\rightarrow$) and standalone `f l` tokens to a down-arrow ($\downarrow$), matching the original SOP semantics. The remapping is restricted to word-boundary matches, so ordinary words containing these letter pairs are left untouched. HTML entities are decoded, escaped formatting characters (e.g.

`_`) are sanitised to prevent Markdown injection errors in downstream prompts, and runs of repeated spaces are collapsed to keep the token count low.

5.3.2 Table Cell Deduplication

A recurring hallucination in layout-aware extraction occurs when overlapping bounding boxes cause cell text to be repeated sequentially (e.g., “Inspect and lubricate HIGH Inspect and lubricate HIGH”). The parser resolves this with a regex-based algorithm (`_deduplicate_table_cells`) that scans Markdown table rows (identified by pipe `|` delimiters), detects any substring that repeats itself consecutively (separated by whitespace), and collapses it to a single instance, without invoking a secondary LLM call.

5.4 Context-Aware Chunking Strategy

Splitting a document arbitrarily mid-sentence destroys semantic meaning, so segment boundaries must respect the document’s own structure. The parser splits strictly at double newlines, reliable paragraph and section boundaries in Markdown, then merges adjacent paragraphs into a rolling buffer capped at 2000 characters. If the final chunk is shorter than 200 characters it is merged backward into the preceding chunk to avoid uninformative microscopic segments. This guarantees that every segment is a well-formed, self-contained block of paragraphs rather than a fragment cut mid-table or mid-sentence.

The sanitised segments are then reassembled in order and saved as a single `.txt` file per document, named after its source (e.g. `SOP_001_OperatingProcedures.txt`).

Note on the two chunking stages. The chunking described above is internal to this layer: its segments are reassembled before the file is written, and their boundaries do not survive into extraction. The extraction pipeline performs its own segmentation on the reassembled text (Section 8.1), with a different size target and, critically, with every content line numbered so that each extracted record can cite the line it came from. The two are independent by design: this layer’s job is to produce faithful text, and the boundary at which a model reads that text is a decision the extraction pipeline owns, because it is the stage that must make provenance checkable.

These files serve as the direct input to both the grid search (Section 7) and the multi-agent extraction pipeline (Section 8), each of which reads the reassembled text subject to a fixed per-call character budget.

6. METHODOLOGY

This chapter details the full research methodology underlying the thesis. It describes the four corpora against which every system is evaluated, the experimental conditions under which extraction is run, the content-first metric introduced in this work together with the evidence establishing what it measures and what it does not, and the protocol that makes every reported figure a mean over repeats rather than a single draw. Individual system components (the document parser, the schema-specified grid, and the multi-agent pipeline) are described in the sections that follow; this chapter focuses on the *how* and *why* of measurement and experimental design rather than the systems themselves.

6.1 Research Design and Experimental Overview

The thesis follows a three-layer experimental design, progressing from raw document ingestion through extraction to evaluation across four corpora. The layers correspond to the major phases of the work:

Layer 1: Document Ingestion and Parsing. Source PDF documents are converted into clean, semantically structured plain text using a vision-aware extraction pipeline built on IBM’s `docling` library (Section 5). This layer is a prerequisite for all downstream extraction steps.

Layer 2a: Schema-Specified Baseline. A grid over ten prompting paradigms and two models, in which the prompt carries the development corpus’s schema, taxonomy, identifier convention and worked examples, establishes what single-prompt extraction achieves when the target schema is known (Section 7). Because such a prompt is written around one corpus, this condition also delimits what that accuracy is contingent upon.

Layer 2b: Multi-Agent Extraction Pipeline. A LangGraph pipeline that receives no schema: segmentation with line numbering, parallel scout agents that copy each document’s own field labels, a corpus-level schema inducer that never sees a document, deterministic assembly, and parallel grounding audit (Section 8).

Layer 3: Evaluation. Every system is scored by $F1_{\text{content}}$ against each corpus’s own ground truth, five repeats per configuration, with the metric’s discriminating power established empirically (Section 6.6).

All experiments rely exclusively on prompt engineering and architectural design decisions. No training, fine-tuning, or weight adaptation of any model was performed. This design reflects two constraints: the size of the annotated ground truths (26–86 rules per corpus) makes task-specific fine-tuning statistically untenable, and the research objective concerns what an off-the-shelf model achieves when given no corpus-specific configuration, a question that fine-tuning on one corpus would foreclose rather than answer.

6.2 Corpora: One Development Corpus and Three External Test Corpora

Extraction is evaluated on four corpora drawn from unrelated industrial domains. One serves as the development corpus, on which the pipeline was built and debugged; the remaining three are external test corpora, seen for the first time at evaluation. Table 2 summarises them.

Table 2: The four evaluation corpora. Only the development corpus was available while the pipeline was being written; the three external corpora were scored without any per-corpus configuration, prompt change, or field mapping.

Corpus	Role	Docs	GT rows	Dominant structure
Production line (iMAKS)	development	4	86	narrative, table, mixed, matrix
Biogas digestion	external test	1	47	parameter table + triggered responses
Cleanroom access control	external test	1	37	role/zone permission matrix
RO desalination	external test	1	26	printed rule codes + intervals

The split is the central design decision of the evaluation. A system reported only on the corpus it was developed against cannot be distinguished from one tuned to those documents, however careful the tuning was avoided in intent. The three external corpora were therefore written to differ from the development corpus in the dimensions that matter to a schema-agnostic extractor: they use different field vocabularies (`bound_low/bound_high` rather than `critLo/critHi`), different rule taxonomies, different identifier conventions (two of them number their rules with annotator-assigned codes that appear nowhere in the source text, one reuses codes printed in the document) and different granularity conventions, discussed in Section 6.5.8.

6.2.1 Development Corpus

The development corpus is the iMAKS (industrial Multi-Agent Knowledge extraction Synthetic) benchmark [5], simulating five two-shift operational days at a nine-station packaging facility. Its synthetic construction ensures that ground-truth annotations are complete and unambiguous; a real industrial SOP corpus would introduce annotation noise and confidentiality constraints orthogonal to the extraction problem studied here. Its four documents span four distinct document structures within a single corpus, which is what makes it a useful development target.

6.2.2 Facility Structure

The simulated facility consists of nine stations organised into six zones: four production stations (`ST01_FILLING`, `ST02_SEALING`, `ST03_LABELLING`, `ST04_PACKAGING`) and five auxiliary stations (`SRV01_SERVERROOM`, `WRH01_WAREHOUSE`, `CHM01_CHEMICALSTORAGE`, `RND01_RDLAB`, `CAF01_CAFETERIA`), with twenty-two physical sensors deployed across them. The station and sensor identifiers matter here only because they are the subjects that extracted rules refer to; this thesis evaluates rule extraction from the documents, not the sensor stream.

6.2.3 SOP Document Corpus and Structural Heterogeneity

The four SOP documents (SOP-001 through SOP-004) are the primary extraction target. Each encodes operational knowledge in a structurally distinct format:

- **SOP-001 (narrative):** Plain procedural prose describing operational conditions, process control actions, and zone-level safety constraints. Rules are embedded in free-form sentences with no table structure or column delimiters. The extraction challenge is purely semantic: the model must identify which sentence fragment constitutes a conditional operational constraint and parse its subject (the sensor or station), predicate (the threshold or condition), and action.
- **SOP-002 (tabular):** A sensor-threshold table with one row per sensor and four numeric threshold columns (CRIT_LO, WARN_LO, WARN_HI, CRIT_HI) plus a measurement unit column. The extraction task is more mechanical than SOP-001, amounting to column-to-field mapping, but OCR artefacts from PDF vision-extraction, unit-suffix handling, and partial rows (sensors with critical but no warning limits) introduce non-trivial failure modes. All 27 `ThresholdRule` ground-truth entries come from SOP-002, and no entry of any other class does.
- **SOP-003 (mixed):** A structured table whose rows encode maintenance procedures and correlated-fault patterns as (trigger condition, required action, priority) triples. The document is classified as *mixed* because it combines the structured tabular layout of SOP-002 with the semantic complexity of SOP-001: each row requires the model to read a multi-clause trigger condition and a multi-step action description. SOP-003 also states cross-station causal rules (RULE-CORR-01 links a sealing motor overload at ST02_SEALING_CUR to a packaging speed anomaly at ST04_PACKAGING_SPD) whose subject is a relationship between two stations rather than a single sensor. Such rules are extracted here as records like any other; whether a downstream system could act on them is outside the scope of this evaluation.
- **SOP-004 (matrix):** A role-zone access-control and occupancy matrix. Rows represent facility zones and columns represent access-permission classes or acknowledgment deadlines. Rules are encoded in individual cells rather than in sentences or table rows, requiring the extractor to correctly associate each cell value with its row label (the zone) and its column label (the permission type or deadline class). Cross-cell inference, where a model borrows a value from an adjacent cell, is the primary failure mode for this document type.

This intentional structural heterogeneity is one of the two experimental challenges. A prompt optimised for narrative prose under-extracts from tabular and matrix documents; a prompt optimised for structured rows produces poor results on prose. A pipeline that must work without being told which structure it faces cannot resolve this by routing to a structure-specific prompt, because the set of structures is not known in advance either. It is resolved instead by asking the extracting agent to copy each document’s own labels rather than to fill a fixed template, so that a table’s column headers, a matrix’s axis labels, and a sentence’s subject all become field names by the same mechanism (Section 8).

6.2.4 External Test Corpora

The three external corpora were held out entirely during development. Each is a single document from a domain unrelated to packaging:

- **Biogas digestion (47 ground-truth rows).** A digester operating procedure combining a parameter table (tag, equipment, measured quantity, unit, low and high bounds) with triggered-response text attached to the same physical rows. Its ground truth splits one printed table row into two annotated rows (one for the limit, one for the response) and numbers them R01, R02, codes that appear nowhere in the document.
- **Cleanroom access control (37 ground-truth rows).** A permission matrix over roles and equipment zones, together with environmental limits and acknowledgement deadlines. Its ground truth annotates one row per matrix cell and numbers them M001, M002.
- **Reverse-osmosis desalination (26 ground-truth rows).** A high-pressure skid procedure whose rules carry identifiers printed in the document itself (OP-ST-001), unlike the other three corpora. Its ground truth leaves the identifier column blank on rows with no printed code, which has consequences for identifier-based scoring discussed in Section 6.5.9.

The three corpora disagree with the development corpus and with each other about what constitutes one rule. That disagreement is not incidental: it is the reason a fixed extraction schema cannot be transferred, and it bounds what any row-matching metric can report (Section 6.5.8).

6.2.5 Ground-Truth Rule Schema

The development corpus provides 86 manually annotated ground-truth rules distributed across four classes (28 operational, 27 threshold, 20 access, 11 maintenance). Its schema is described here in full because it is the corpus against which field-level placement is verified (Section 9.2); the three external corpora use their own schemas, with different field names, different class vocabularies and different identifier conventions, and none of them is supplied to the pipeline. Each development-corpus rule is a structured record with thirteen fields:

- **Identifier:** `ruleId`, a human-assigned naming-convention string (e.g., RULE-ST01-01, MAINT-04, RULE-THR-ST02-TMP-CRIT).
- **Categorical fields:** `class`, `station`, `sensor`, `sensorType`, `severity`, `unit`.
- **Numeric threshold fields:** `critHi`, `warnHi`, `warnLo`, `critLo`.
- **Text fields:** `condition`, `action`.

The four rule classes and their semantic roles are:

- **ThresholdRule**: numeric alarm boundaries for a specific sensor at a specific station, sourced primarily from SOP-002. The key operational content is the combination of sensor identity and numeric threshold values.
- **OperationalRule**: condition-action rules governing process control (e.g., “if filling flow drops below 105 L/min, halt the filling cycle”), sourced primarily from SOP-001. The key operational content is the textual condition and action.
- **MaintenanceRule**: scheduled or reactive maintenance procedures triggered by sensor fault patterns, concentrated in SOP-003. The key operational content is the trigger condition and the prescribed maintenance procedure.
- **AccessRule**: zone occupancy limits, access permissions, and acknowledgment deadlines, sourced from SOP-004. The key operational content is the zone identifier and the access or timing constraint.

A key characteristic of the annotations is that `ruleId` values are annotator-assigned conventions that frequently do not appear verbatim in the source SOP text. An extraction system cannot reproduce the exact annotation string without prior knowledge of the annotation convention; this is the fundamental motivation for the content-first evaluation metric defined in Section 6.5. These 86 annotated rules are the ground truth against which every development-corpus figure in this thesis is scored; the external corpora are scored against their own, described in Section 6.2.4.

6.3 Experimental Conditions

6.3.1 Inference Endpoint

Every agent role is served by Ollama Cloud through its OpenAI-compatible API, configured by `OLLAMA_BASE_URL` and `OLLAMA_API_KEY`. Nothing in the pipeline depends on the endpoint being hosted rather than local: the same code runs against any OpenAI-compatible server, and the model names are the only endpoint-specific values. The choice does, however, have one methodological consequence that shapes how every result in this thesis is reported, developed in Section 6.3.3: a hosted endpoint batches requests with other traffic, and the resulting variation cannot be suppressed by a seed.

Requests use a 600 s timeout with up to three attempts per call and a 15 s base backoff between them, and the scout and audit fan-outs issue at most four concurrent requests. The output budget is 32 768 tokens per call, generous because reasoning models bill their internal deliberation against the same budget, and a reply truncated mid-object loses every record after the cut.

6.3.2 Language Models

Two models are evaluated in the grid search (Section 7): `gemma4:31b` and `nemotron-3-nano:30b`. The multi-agent pipeline assigns `gemma4:31b` to all three agent roles: scout, inducer, and auditor.

Assigning one model to every role is a deliberate choice supported by the grid rather than a simplification. Over 100 grid runs, rescored with the metric of Section 6.5, `gemma4:31b` scored higher than

nemotron-3-nano:30b on all ten prompting paradigms, and by a margin comparable to the spread between paradigms themselves: the spread across paradigms within gemma4:31b is 0.132 (0.814–0.946), while the same paradigm across the two models differs by up to 0.128. Model choice therefore matters about as much as prompting strategy, and there is no measured basis for handing any role to the weaker model. Role independence between the extracting and auditing agents (an auditor that does not share the scout’s failure modes) is available through a command-line override and is not used by default, because the only alternative measured here scored lower on every paradigm and varied more from run to run; an auditor that unreliable removes more correct records than incorrect ones.

Note on reasoning suppression. Some models expose a `reasoning_effort` parameter that gates an internal deliberation pass. gemma4:31b performs no such pass by default, and passing any effort value *enables* one; the parameter is therefore omitted entirely for that model rather than set to a nominal value.

6.3.3 Temperature, Seed, and the Limits of Reproducibility

All conditions run at temperature $T = 0$ with seed 42. Neither setting is sufficient to make a run reproducible on a hosted endpoint, and this thesis does not claim that it is.

Greedy decoding fixes the token choice given a probability distribution, but it does not fix the distribution. Requests to a shared endpoint are batched together with unrelated traffic, the batch composition varies between calls, and floating-point summation order varies with it. Over a short reply the resulting numerical differences are absorbed; over a long structured generation they compound until two tokens exchange rank, after which the two replies diverge outright. A fixed seed does not address this, because the randomness is not in the sampler.

Two consequences follow, and both are load-bearing for the results reported later. First, every configuration is run five times and reported as a mean with its standard deviation; a single run is one draw, and two configurations whose intervals overlap are tied rather than ranked, however different their means appear. Second, exact replay is provided by a response cache keyed on SHA-256(model,messages) rather than by a seed: a warm cache reproduces a previous run exactly, and any change to a prompt, a model, or a document changes the key and forces a real call, so the cache cannot silently serve a stale answer. The cache is disabled automatically whenever repeats are requested, since repeated identical requests would otherwise be served from it and the measured spread would be zero, not because the pipeline is stable, but because it never ran again.

6.3.4 Chunking and the Output Budget

Documents are not passed to a model whole. Segmentation packs blank-line-delimited blocks into chunks of approximately 3 500 characters (Section 8.1), so input length is bounded by construction and no document-level truncation limit is required; a corpus of any size is processed as more chunks rather than as a truncated document.

The output budget is set to 32 768 tokens per call. The allocation is deliberately generous for two reasons. Extracting every rule from a dense chunk as a JSON array can require several thousand output tokens, and reasoning models bill their internal deliberation against the same budget, so a model may consume most of

the allowance before its answer begins. A reply cut off at the budget loses every record after the cut, which is why truncation is detected from the response’s finish reason and surfaced as a warning rather than silently absorbed (Section 8.7).

6.4 Evaluation Metric Design: Motivation and Framework

6.4.1 Limitations of Standard Exact-Match Metrics

The conventional approach to evaluating information extraction systems is to compute F1 against a ground-truth annotation using exact string matching. Applied to rule extraction, the natural analogue is identifier-based F1: a predicted rule is a true positive if and only if its `ruleId` exactly matches a ground-truth entry. Identifier-based F1 suffers from a fundamental ceiling in this setting that makes it unsuitable as an evaluation criterion, and the ceiling is a property of the annotation rather than of any system.

Ground-truth identifiers are typically annotator-assigned naming conventions absent from the source text. Three of the four corpora used here number their rules with codes that appear nowhere in the documents (RULE-ST01-01, R01, M001) while the fourth reuses codes the document itself prints. An extraction system cannot reproduce an invented convention without being told what it is, regardless of how accurately it captures the operational content, so identifier agreement on those three corpora is exactly zero for any extractor whatsoever. This is why identifier agreement is reported in this thesis as a diagnostic of the annotation convention rather than as a score (Section 6.5.9).

The consequence is a systematic evaluation blind spot: a system that extracts a semantically perfect rule, with the correct sensor, station, threshold values, and action text, may receive a false-negative because it assigned a plausible but non-matching identifier (e.g., RULE-ST01-TEMP-CRIT instead of RULE-THR-ST01-TMP-CRIT). Conversely, a system that hallucinated an incorrect threshold value but guessed the correct identifier would receive a true positive. Identifier-based F1 therefore conflates extraction quality with naming-convention alignment, which are largely independent properties.

6.4.2 Design Principles for $F1_{\text{content}}$

The primary metric introduced in this thesis, $F1_{\text{content}}$, is designed around four principles:

1. **Content primacy.** The evaluation must reflect whether the extracted rule conveys the same operational constraint as the ground-truth rule. Identifier strings are deliberately excluded from the scoring function.
2. **Independence from both sides’ field names.** The metric must not require a correspondence between the ground truth’s field names and the extractor’s, because no such correspondence exists and constructing one per corpus would reintroduce the per-domain configuration the pipeline exists to remove. This rules out any weighting scheme defined over named fields (including class-dependent weights) and is why the comparison operates on whole-row text rather than on aligned cells (Section 6.5.3). The cost of this choice is real and is stated rather than hidden: the metric cannot tell whether a value sits in the right slot, which Section 10.2 quantifies and Section 9.2 addresses by separate means.

3. **Optimal one-to-one assignment.** Evaluation must use a globally optimal bipartite matching between the predicted and ground-truth rule sets to prevent double-counting and to produce principled precision and recall values.
4. **Partial credit with a hard TP threshold.** Rules that are mostly but not perfectly correct should receive partial credit via a continuous content-agreement score. The final true-positive decision should nonetheless be based on a threshold, to avoid inflating F1 through accumulation of many low-quality near-matches.

6.5 The Metric: Content Agreement over Optimally Assigned Pairs

The evaluation compares two sets of records that share no vocabulary. A ground-truth row names its fields `critLo`, `bound_low`, or `min_bound` depending on which corpus wrote it; a predicted record names its fields whatever the source document called them. No mapping between the two exists, and constructing one by hand for each corpus would reintroduce exactly the per-domain configuration the pipeline exists to remove. The metric therefore aligns nothing by name.

6.5.1 The Procedure End to End

Before the individual components are defined, Algorithm 3 states the whole procedure, so that each subsequent subsection can be read as an expansion of one numbered step rather than as an isolated definition.

Table 3: Scoring one prediction file against one ground truth. Steps 1–4 are independent of the acceptance threshold, which is why a threshold sweep (Section 6.6) reuses a single execution of them.

Input:	ground-truth rows G , predicted records P , threshold τ , weights $w_{\text{sem}}, w_{\text{num}}$
---------------	---

1. Detect and remove the identifier column and any provenance columns from G ; remove the pipeline’s fixed bookkeeping columns from P . Detection is structural (Section 6.5.4).
2. Collapse every remaining row of G and record of P into a text blob by joining its surviving values with “ | ”.
3. Extract the numbers appearing anywhere in each blob (Section 6.5.5).
4. For every pair $(g, p) \in G \times P$, compute $\text{agree}(g, p)$ by Equation 1, forming an $|G| \times |P|$ matrix.
5. Solve the matrix as a linear assignment problem, maximising the total agreement over all one-to-one pairings. This yields $\min(|G|, |P|)$ pairs.
6. Count a pair as a true positive when $\text{agree}(g, p) \geq \tau$.
7. Set $\text{FP} = |P| - \text{TP}$ and $\text{FN} = |G| - \text{TP}$, then compute precision, recall and $F1_{\text{content}}$.
8. Emit one audit row per decision (matched pairs, unpaired ground-truth rows, unpaired records) recording every intermediate quantity (Section 6.7).

Two properties of this procedure are worth stating explicitly because they are frequent sources of confusion. First, *no field is ever compared to a field*: the comparison operates on whole-row blobs, so a ground truth calling a quantity `bound_low` and a record calling it `l1` are never required to agree on the name. Second, *the comparison is not lexical*: two blobs sharing no words can still be paired if their sentence

embeddings are close, and two blobs sharing many words can fail to reach the threshold. The word-level overlap reported in the audit trail is a diagnostic for the reader and plays no part in steps 4–7.

6.5.2 A Worked Example

The following is a real comparison, taken verbatim from the audit trail for run 1 of the biogas corpus, and is reproducible by opening that file.

A pair that is counted. Ground-truth row R04 and predicted record SOPBDG003MID-002 produce these two blobs after step 2:

```
GT AlarmControlResponse | DGT-101 | pH | <6.5 at any time |
DCS stops feed pump PMP-201, raises Critical alarm
DGT-101 pH Low-Low | Critical | 6.5 | pH
PRED StaticLimitAndTriggeredResponse | If pH <6.5 at any time, DCS stops
feed pump PMP-201 and raises Critical alarm "DGT-101 pH Low-Low". |
Primary Digester 1 | 7.5 | 6.8 | pH | DGT-101 | -
```

Note what the two sides disagree about. The ground truth calls this rule an `AlarmControlResponse`; the record calls it a `StaticLimitAndTriggeredResponse`, a label from a vocabulary the inducer derived for this corpus and which the ground truth never uses. The record also carries the equipment name and the normal operating band (6.8–7.5) that the ground-truth row omits, and the two sides order their fields differently, because each follows its own column order. Under exact matching on any field this pair fails. The metric proceeds as follows.

Step 3, numbers. The ground truth yields $N_g = \{-101, 6.5, -201, -101, 6.5\}$ and the record $N_p = \{-101, 1, 6.8, 7.5, 6.5, -201, -101\}$. The negative values are asset tags: the number extractor reads the hyphen in DGT-101 as a minus sign, so the tag contributes -101 on both sides (Section 6.5.5 discusses this).

Step 4, agreement. Each ground-truth number finds an exact counterpart in the record, so every per-number score in Equation 2 is 1.000 and $\text{numOverlap} = 1.000$. The sentence embeddings give a cosine of 0.855. Therefore

$$\text{agree} = 0.6 \times 0.855 + 0.4 \times 1.000 = 0.913,$$

which is the value the audit trail records, along with the per-number pairing (6.5→6.5 (1.000), and so on) that produced the numeric term.

Steps 5–6. The assignment selects this pair, and $0.913 \geq 0.6$, so it is a true positive. For orientation the audit also notes that the two blobs share 18 words at a Jaccard overlap of 0.667, a number that played no part in the decision.

A pair that is rejected. The assignment is exhaustive over $\min(|G|, |P|)$ pairs, so it pairs rows even when no good partner remains. Ground-truth row R15 was assigned to record SOPBDG003MID-036:

```
GT AlarmControlResponse | GSH-401 | Level | <20% |
DCS inhibits CHP gas supply, Warning alarm active | Warning | 20.0 | %
PRED TriggeredResponse | A warning sign interlocked with the level
```

transmitter shall illuminate at the access gate.

Both rows concern a level signal and both mention a warning, which is why the assignment paired them once better partners were taken. But the cosine is only 0.476, and the record contains no number at all, so $\text{numOverlap} = 0.000$ against a ground-truth row that states a 20% threshold. The blended score is $0.6 \times 0.476 + 0.4 \times 0.000 = 0.286$, far below $\tau = 0.6$. The pair is recorded as REJECTED (below threshold) and counts as a false negative for R15 and a false positive for the record. This is the correct outcome, and it illustrates why the numeric term is load-bearing: on semantic similarity alone the two rows are not obviously unrelated, and it is the absence of the threshold value that decides the case.

From decisions to the reported figure. That run produced 37 records against 47 ground-truth rows. The assignment formed 37 pairs, of which 33 cleared the threshold and 4 did not, leaving 10 ground-truth rows unpaired for want of a record. Step 7 then gives

$$\text{TP} = 33, \quad \text{FP} = 37 - 33 = 4, \quad \text{FN} = 47 - 33 = 14,$$

$$P = \frac{33}{33+4} = 0.892, \quad R = \frac{33}{33+14} = 0.702, \quad F1_{\text{content}} = \frac{2 \times 0.892 \times 0.702}{0.892 + 0.702} = 0.786.$$

Every quantity in this derivation is written to `f1_derivation_<corpus>.csv` with the substitution shown, so the arithmetic can be checked without re-running anything.

6.5.3 Why There Is No Field-by-Field Comparison

The ground truth has fields, and the natural expectation is that scoring compares them one by one: the ground truth's `bound_low` against the record's corresponding value, its `action` against the record's `action`, and so on. No such comparison is performed, and this subsection states precisely why and what is done instead, because it is the single most consequential design decision in the evaluation.

The obstacle. A field-by-field comparison requires a correspondence between the two sides' field names. That correspondence does not exist. The biogas ground truth names its bounds `bound_low` and `bound_high`; the desalination ground truth names them `min_bound` and `max_bound`; the development corpus names them `critLo`, `warnLo`, `warnHi` and `critHi`. On the predicted side the names are whatever each source document printed: `l1` and `h1` in the example below. Building the correspondence means a human reading both schemas and writing a mapping per corpus, which is exactly the per-domain configuration the pipeline exists to remove; a metric requiring it could not be applied to a new facility any more than a schema-specified prompt could.

What is done instead. Both sides are flattened. Every surviving column's *value* is concatenated in column order; the *names are discarded entirely*. The comparison then happens between two flat strings. For the ground-truth row R04 of the worked example (Section 6.5.2), the twelve stored columns reduce as follows:

<code>id</code>	<code>= R04</code>	<code>-> dropped (identifier)</code>
<code>category</code>	<code>= AlarmControlResponse</code>	<code>-> kept</code>

```

asset      = DGT-101                -> kept
tag        =                        -> dropped (empty)
parameter  = pH                     -> kept
condition  = <6.5 at any time       -> kept
action     = DCS stops feed pump ... -> kept
severity   = Critical               -> kept
bound_low  = 6.5                    -> kept
bound_high =                        -> dropped (empty)
unit       = pH                     -> kept
source     = SOP-BDG-003            -> dropped (provenance)

```

leaving the blob AlarmControlResponse | DGT-101 | pH | <6.5 at any time | DCS stops feed pump PMP-201, ... | Critical | 6.5 | pH.

The predicted record is flattened by the identical rule. It has its own columns, discovered from the source document rather than declared anywhere, and for this corpus there are 28 of them, one for every field any record in the corpus was observed to carry. The record matched to R04 populates eight:

```

id          = SOPBDG003MID-002 -> dropped (bookkeeping)
category    = StaticLimitAndTriggeredResponse -> kept
action_remarks = If pH <6.5 at any time, DCS ... -> kept
equipment    = Primary Digester 1 -> kept
hl          = 7.5                -> kept
ll          = 6.8                -> kept
parameter   = pH                 -> kept
tag         = DGT-101            -> kept
unit        = -                  -> kept
(19 further discovered columns, empty for this record -> dropped)
source_file  = SOP_BDG_003_...txt -> dropped (bookkeeping)
source_span  = ...:L21#1 | ...    -> dropped (bookkeeping)

```

Note that the two sides express the same bound under different names (the ground truth's bound_low against the record's ll, taken from the document's own column header) and that neither name survives into the blob.

The consequence, stated plainly. After flattening, the metric no longer knows that 6.5 was a bound_low. It knows only that 6.5 appears in the ground-truth row and that some number close to it appears in the record. Two things follow, and both are real:

- **The benefit.** A record is not penalised for naming a field differently from the annotator, for splitting one field into two, or for assigning a category from a vocabulary the ground truth never uses. In the worked example the two sides disagree on the category outright (AlarmControlResponse against StaticLimitAndTriggeredResponse, a label the inducer derived for this corpus) and the pair is still correctly counted, because the operational content agrees.

- **The cost.** A value placed in the *wrong* field produces the same token in the blob as one placed correctly. If a record attached 6.5 as an upper bound rather than a lower one, the flattened strings would be indistinguishable. The metric therefore checks that the right values are present, not that each is bound to the right slot. This is not a conjecture: permuting quantities between records (so that every rule carries another rule’s bound) costs -0.007 F1 (Table 5), which is to say the metric does not detect it at all. The blind spot is carried into the limitations as Section 10.2.

The design is therefore a deliberate trade: field-name independence, which is what makes cross-corpus evaluation possible at all, is bought at the price of field-level precision. A metric that recovered the latter without reimposing a per-corpus schema mapping would be a genuine improvement, and none is offered here.

6.5.4 Blob Construction (Steps 1–2)

Each ground-truth row and each predicted record is collapsed into a single text blob: every column’s value, whatever the column is called, joined in order with “ | ”. Two kinds of column are removed first, and both are identified structurally rather than from a list of known names, so that a corpus using an unfamiliar column name still resolves correctly:

- **The identifier column.** Every column is scored by $u - \bar{\ell}/200 + h$, where u is the fraction of non-empty values that are distinct, $\bar{\ell}$ is the mean value length, and h is a bonus of 0.05 if the column name is one of `id`, `ruleId`, `rule_id`. Columns with $u < 0.9$ are disqualified outright, and the highest-scoring survivor is taken. The criterion encodes the statistical signature of an identifier (distinct on almost every row, and short) while the name bonus is small enough to break a near-tie and never to decide one alone. The column is excluded from the blob because matching identifiers measures naming convention rather than content (Section 6.5.9).
- **Provenance columns.** A column is provenance if it holds the same value on every row, or if it has at most $\max(6, \lfloor 0.1 |G| \rfloor)$ distinct values *and* each of those values is at most 24 characters, contains no space, and contains a digit: the shape of a document code rather than of rule content. The second test exists because a corpus of four documents gives its source column four values, which the constant-value test misses; a genuine categorical field such as `severity` fails it because its values are words. On all four corpora exactly one column is removed by this test (`source`) and no content column is affected.

Both thresholds above are declared parameters rather than inline constants, and are listed with the other free parameters in Table 4.

The predicted side drops the pipeline’s own fixed bookkeeping columns by the same principle. In particular the recorded source span, which names the line a record was read from, is excluded: no ground truth carries such a column, so every token it contributes is unmatchable by construction and can only dilute the comparison.

6.5.5 Agreement Between Two Blobs

For a ground-truth row g and a predicted record p with blobs b_g, b_p :

$$\text{agree}(g, p) = w_{\text{sem}} \cdot \cos(\text{SBERT}(b_g), \text{SBERT}(b_p)) + w_{\text{num}} \cdot \text{numOverlap}(g, p), \quad (1)$$

with $w_{\text{sem}} = 0.6$ and $w_{\text{num}} = 0.4$. The semantic term is cosine similarity of sentence embeddings [20] (all-MiniLM-L6-v2), which is what allows `EnvironmentalLimit | TMP | 175 | 195` to align with `ThresholdRule | TMP` although neither side knows the other’s vocabulary.

The numeric term. The numeric term exists because sentence embeddings barely distinguish 175 from 195, and a rule with the wrong bound is wrong. Numbers are extracted from a blob by the pattern `-\?d+\.\?\d*` applied to the whole string, so a value is found wherever it was written and whichever column held it. Writing N_g and N_p for the numbers so obtained:

$$\text{numOverlap}(g, p) = \frac{1}{|N_g|} \sum_{x \in N_g} \max_{y \in N_p} \max\left(0, 1 - \frac{|x - y|}{\max(|x|, |y|)}\right). \quad (2)$$

The score for one ground-truth number is therefore 1 for an exact match and decays with relative error, reaching 0 only when the nearest predicted value is of opposite sign or more than twice as far from zero. The per-number pairing that produced the average is written to the audit trail, so a reader can see which predicted value was credited against which ground-truth value and how close it had to be.

One artefact of this extraction is visible in the worked example of Section 6.5.2 and is declared rather than silently corrected: a hyphen inside an asset tag reads as a minus sign, so `DGT-101` contributes `-101`. Such values are about 15% of all numbers extracted from these ground truths. They are counted, because a shared asset tag is genuine evidence that two rows concern the same equipment, and because excluding them changes reported figures; the choice is exposed as a parameter (Table 4) so that it is auditable rather than buried in a regular expression.

When a ground-truth row contains no numbers at all the numeric term is undefined rather than zero, and the agreement is the semantic cosine alone. This asymmetry is reported in the audit trail rather than hidden, because it means a row without numbers must reach the acceptance threshold on semantic similarity unaided, while a row with numbers may clear it on a weaker semantic match carried by numeric agreement.

6.5.6 Optimal Assignment and $F1_{\text{content}}$

Pairing is global, not greedy. The full $|G| \times |P|$ matrix of agreement scores is solved as a linear assignment problem by the Hungarian algorithm [18], maximising total agreement over all one-to-one pairings. Each ground-truth row is therefore credited to at most one record and each record to at most one row, so a single well-formed record cannot be counted as satisfying three different rules.

A pair counts as a true positive when its agreement reaches the threshold $\tau = 0.6$. Writing n_G and n_P for the number of ground-truth rows and predicted records:

$$TP = |\{(g, p) \text{ assigned} : \text{agree}(g, p) \geq \tau\}|, \quad FP = n_P - TP, \quad FN = n_G - TP, \quad (3)$$

with precision, recall and $F1_{\text{content}}$ following in the usual way. Note that the assignment maximises *total* agreement rather than the *count* of pairs above threshold; the two can differ in principle, and the conventional choice is taken here.

6.5.7 Free Parameters

Every constant the evaluator uses is declared in one configuration object rather than written inline, because each is a researcher degree of freedom: a number a human chose, which could in principle have been chosen by looking at results on the corpus being scored. Collecting them makes them countable, and every one is overridable from the command line so that a reported figure can be shown stable across a range rather than dependent on one setting.

Table 4: Every free parameter of the evaluation, its value, and what it governs. The defaults are conventional starting points, not values fitted to any ground truth in this study; Section 6.6 reports where the operating point sits on the threshold curve.

Parameter	Value	Governs
τ (threshold)	0.6	agreement required for an assigned pair to count as a true positive
w_{sem}	0.6	weight on the SBERT cosine term
w_{num}	0.4	weight on the numeric-overlap term
embedding model	all-MiniLM-L6-v2	sentence embeddings; English-only, matching the corpora
id uniqueness minimum	0.9	distinct-value fraction below which a column cannot be the identifier
id length penalty divisor	200	how strongly long values disqualify a column from being the identifier
id name-hint bonus	0.05	tie-break nudge for a conventional identifier name
provenance distinct floor	6	maximum distinct values for a document-label column
provenance distinct frac.	0.1	row-fraction alternative to the floor, whichever is larger
provenance max value len.	24	maximum length of a value that still looks like a document code
drop tag-derived numbers	off	whether to discard negative values arising from hyphenated asset tags

6.5.8 The Precision Ceiling Imposed by Granularity

Equation 3 has a consequence that must be stated before any result is read. The assignment forms at most $\min(n_G, n_P)$ pairs, so whenever a system emits more records than the ground truth has rows, the surplus are false positives *by construction*: they had no row left to be assigned to, whether or not their content is correct. Precision is therefore capped at

$$\text{precision} \leq \frac{\min(n_G, n_P)}{n_P}. \quad (4)$$

This is not a defect of the implementation but a property of one-to-one row matching, and it interacts directly with the corpora’s disagreement about what constitutes one rule (Section 6.2.4). A system whose granularity differs from the annotator’s is penalised for the difference, in either direction.

It is worth being precise that the mismatch does *not* reduce to the pipeline being uniformly more verbose. On the development corpus it runs both ways at once. For threshold rules the ground truth is the *finer* of the two: its 22 threshold sensors are annotated as 50 rows (ten sensors carry three rows each, eight carry two) because one printed table row is split into an operational band, a critical trip, and a threshold record. The pipeline emits one row per sensor there, and is therefore coarser. Elsewhere (matrix cells, inter-station dependencies) the pipeline is finer. The two effects net out to more records than rows overall, but no single collapse or split rule could reconcile them, which is why none is applied. The ceiling is reported alongside the precision it caps in every results table, since a precision of 0.668 against a ceiling of 0.756 and the same figure against a ceiling of 1.0 describe entirely different systems.

6.5.9 Identifier Reuse as a Diagnostic, Not a Score

Identifier agreement is reported, but deliberately not as a precision/recall/F1 triple. What it measures is whether the corpus’s annotator reused codes printed in the source document or invented their own numbering, which is a property of the annotation and not of the system under test. A ground truth numbering its rules R01 and M001 (strings appearing nowhere in the document) yields zero identifier agreement for any extractor whatsoever, while one reusing printed codes such as OP-ST-001 yields high agreement for any extractor that copies what it reads.

Expressed as an F1 and placed beside $F1_{\text{content}}$, this quantity invites the reading that the system’s honest score is near zero on the corpora where it is zero. It is therefore reported as a fraction of the ground truth’s distinct identifiers recovered verbatim, labelled as a diagnostic, with the distinct-identifier count given alongside, necessary because a ground truth leaving its identifier column blank on some rows has fewer identifiers than rows, and the fraction would otherwise be read against the wrong denominator.

6.6 Establishing That the Metric Measures Anything

A metric built on sentence embeddings can report a respectable score while discriminating nothing, because embeddings place any two documents of the same genre at a non-trivial cosine similarity. A reported $F1_{\text{content}}$ of 0.7–0.8 is therefore not self-interpreting, and this section establishes what it is worth by measurement rather than by argument. All figures come from the same evaluator that produces the headline results, run over the same five repeats per corpus, with deterministic seeded perturbations.

6.6.1 The Null Case

The decisive test is whether the metric collapses when the predictions cannot possibly be right. Each corpus’s predictions are scored against every *other* corpus’s ground truth: nothing is correct, so a metric reading

content must fall towards zero, while one reading genre similarity will not.

Pooled per corpus, the null score is 0.019–0.113, far below every reported figure. The worst individual pairing is higher and must be reported as such: cleanroom predictions scored against the development corpus’s ground truth reach 0.274, and biogas predictions against the same ground truth 0.213. Both involve the largest and most varied ground truth, where an assignment may select its best 36 of 86 rows. The practical consequence is that reported figures should be read against a floor of roughly 0.27 for a large ground truth rather than against 0.

6.6.2 Perturbation Tests

Each perturbation damages one aspect of the predictions, so the score it costs is attributable to that aspect. Table 5 reports the mean $F1_{\text{content}}$ lost across the four corpora.

Table 5: Mean $F1_{\text{content}}$ lost under targeted degradation of the predictions. A perturbation that costs nothing marks something the metric does not check, and is reported as a limitation rather than omitted.

Perturbation	F1 lost	Interpretation
Every numeric value corrupted	0.300	bound values are genuinely verified
Numeric values permuted between records	−0.007	value-to-rule binding is <i>not</i> checked
Category label permuted between records	0.009	the discovered label carries no weight
Token order scrambled (vs. control)	0.000	word order is <i>not</i> read

The first row is the one that licenses the metric’s use for rule extraction: replacing every number with a different number costs 0.300 on average, and up to 0.426 on the corpus where it costs most. Bound values are not decoration in this score.

The remaining three rows are limitations, and together they are more restrictive than the first row alone suggests. Taking another record’s quantities while keeping one’s own subject and prose (the failure in which the right rule carries the wrong bound) costs nothing at all (−0.007): the assignment simply re-pairs, recovering by reallocation what the corruption removed. Permuting the discovered category label between records likewise costs nothing (0.009), because after flattening the label is one column among several dozen. And scrambling token order, measured against an order-preserving control that isolates the cost of the rewrite itself, costs 0.000.

The honest summary is therefore narrow: $F1_{\text{content}}$ verifies that the right *values* are present somewhere in the extracted set, and verifies essentially nothing else: not which record holds them, not the label attached to them, not the order in which they are written. It is a set-membership measure over values, and should be read as one. This also explains why the numeric term of Equation 1 is load-bearing: it supplies the only discrimination the metric reliably has.

6.6.3 Where the Operating Point Sits

The acceptance threshold $\tau = 0.6$ is a human choice and therefore a researcher degree of freedom. What matters is not that it is defensible in the abstract but where it sits on the curve. Sweeping τ from 0.4 to 0.9

shows that it is a strict local maximum on *no* corpus: there is no corpus where 0.6 scores better than both 0.55 and 0.65, which is the shape a threshold chosen to flatter the system would have. On the development and cleanroom corpora the curve is declining, so every lower threshold would report a higher figure. On biogas and desalination it is flat: 0.806 at both 0.55 and 0.6, and 0.820 across 0.55–0.65 respectively, meaning the reported figure there is insensitive to the choice.

One corpus must nonetheless be reported with its curve rather than as a single number. Cleanroom moves 0.213 in F1 across one 0.05 step near the operating point, whereas the other three move between 0.000 and 0.052. Relatedly, the ranking of corpora by $F1_{\text{content}}$ is not stable across the sweep, so no claim that one corpus is intrinsically harder than another is supported by these results.

6.7 Reviewability: The Per-Record Audit Trail

A summary figure cannot be checked. Alongside every score the evaluator emits one row per comparison decision: every assigned pair, then every ground-truth row and every record left unpaired. Table 6 lists what each row records. A separate file writes out the counts and the substituted precision, recall and F1 arithmetic per run, as shown at the end of Section 6.5.2.

Table 6: Columns of the per-record audit trail. Everything above the rule is used by the metric; everything below it is reported for the reader and plays no part in any score.

Column	Content
decision	TP, REJECTED (below threshold), UNPAIRED GT, or UNPAIRED RECORD
counts_as	how the decision enters Equation 3
gt_row, gt_id	the ground-truth row and its identifier
pred_row, pred_id	the paired record and its identifier
semantic_cosine	the SBERT term of Equation 1
numeric_overlap	the numeric term, or blank when the row has no numbers
combined_score	the blended agreement
score_formula	the arithmetic written out, e.g. $0.6 \cdot 0.855 + 0.4 \cdot 1.000 = 0.913$
threshold	the τ in force
numeric_detail_gt_to_pred	which predicted number was credited against which ground-truth number, with its score
gt_numbers, pred_numbers	the extracted number sets
runner_up_score, margin_over_runner_up	the best score obtainable from any <i>other</i> record, and the margin
gt_text_compared, pred_text_compared	the two blobs, verbatim
word_overlap_jaccard	lexical overlap of the two blobs (<i>diagnostic</i>)
shared_words	tokens present in both (<i>diagnostic</i>)
gt_only_words, pred_only_words	tokens unique to each side (<i>diagnostic</i>)

Two properties make this an audit rather than a second implementation. First, the trail is emitted from the same assignment object the metric is computed from, so it cannot drift from the reported figure; recomputing

F1 from the audit files alone reproduces every reported value exactly. Second, each ground-truth row’s *runner-up* is recorded (the best score obtainable from any other record) so a reader can see whether a pairing was decisive or a near-tie. Across the 927 assigned pairs of all twenty runs, 73% of rows receive their own best-scoring record and 27% yield it to another row that the global optimum needs it for; 18% of pairs sit within ± 0.05 of their runner-up and should be read as close calls rather than confident matches.

For orientation the audit also reports a word-level view of the same two blobs (shared tokens and Jaccard overlap) explicitly marked as diagnostic and excluded from the score. It is included because the first question asked of any semantic metric is whether it is really comparing text word by word, and the answer is only convincing when the word-level picture is visible next to the score it is being distinguished from.

7. SCHEMA-SPECIFIED EXTRACTION: THE PROMPTING GRID

This section establishes what single-prompt extraction achieves when the target schema *is* known, and, equally important, what that achievement is contingent upon. The grid serves two purposes in this thesis. It selects the prompting paradigm carried into the multi-agent pipeline’s scout agents (Section 8.2), and it provides the contrast of Section 9.3 against which the cost of requiring no schema is measured.

7.1 What the Prompt Is Given

The distinguishing property of every condition in this grid is that its prompt carries the development corpus’s answer format. Specifically, the prompt supplies:

- the exact output schema, naming all thirteen ground-truth fields (`ruleId`, `class`, `station`, `sensor`, `sensorType`, `condition`, `action`, `severity`, `critHi`, `warnHi`, `warnLo`, `critLo`, `unit`);
- the rule taxonomy, naming the ground truth’s classes explicitly and instructing the model which fields to populate for each;
- the identifier convention, by example (`RULE-ST01-01`, `RULE-THR-ENC07-LUX-CRIT`);
- worked examples of complete output records, using real station and sensor identifiers from the benchmark family;
- an explicit column-to-field mapping for the threshold tables (`CRIT_LO` $\rightarrow$ `critLo`, and so on).

This is a strong and entirely legitimate configuration for a system deployed on a corpus whose schema somebody has already written down. It is reported here as such. What it cannot do is transfer: applying any of these conditions to the biogas, cleanroom, or desalination corpora requires a human to read those documents and rewrite all five elements above. The grid therefore measures schema-specified extraction, not general extraction, and Section 9.3 treats it accordingly.

7.2 Models and Experimental Parameters

The grid evaluates two models, `gemma4:31b` and `nemotron-3-nano:30b`, across ten prompting paradigms with five repeats per condition, for 100 runs in total. Both are served through the same OpenAI-compatible endpoint as the multi-agent pipeline (Section 6.3.1), so the two systems differ in what their prompts contain and in their architecture, not in the model or the serving stack.

Network resilience is handled by an exponential backoff mechanism allowing up to three attempts per call, backing off from a 15-second base delay. For all paradigms except one the temperature is 0 with a fixed seed of 42; the self-consistency paradigm uses a temperature of 0.3 across three independent inner runs by construction, since it requires diverse reasoning paths. As established in Section 6.3.3, neither setting makes a run bit-reproducible on a hosted endpoint, which is why every condition is reported as a mean over five repeats with its standard deviation.

7.3 Prompting Paradigms and Complexity Levels

To systematically assess how LLMs interpret industrial logic, the grid search categorises prompting strategies into four distinct complexity levels:

- **Level 0 (Direct Extraction):** Includes the naive approach, which enforces a strict JSON schema without additional constraints. The `few_shot_static` paradigm enhances this by providing two concrete examples one `OperationalRule` and one `ThresholdRule` illustrating the required output format and numeric field mapping. Finally, the `graph_informed` strategy introduces mandatory constraints, forcing the LLM to map its outputs to specific sensor naming conventions (e.g., `ST01_FILLING_TMP`).
- **Level 1 (Reasoning Before Action):** Introduces intermediate cognitive steps before JSON generation. The `cot_basic` paradigm requires standard step-by-step reasoning, while `cot_structured` enforces a strict five-step analytical process: (1) enumerate all stations and zones, (2) identify the sensors and measurement types per station, (3) extract threshold conditions with their numeric values, (4) identify causal dependencies between stations, and (5) identify access-control, occupancy, and maintenance rules. The `pre_act` paradigm requires the model to summarise the document's structure, the categories of rules present, and any cross-section dependencies before producing the extraction.
- **Level 2 (Iterative Refinement and Ensemble Voting):** Incorporates multi-turn logic and ensemble strategies. The `self_consistency` paradigm runs three independent extractions at temperature 0.3 and then aggregates the results through a score-based voting mechanism (Section 7.4). Both `reflexion` and `reflexion_guided` use the `graph_informed` system prompt as their conversational base, inheriting its mandatory sensor-naming constraints, whereas `self_consistency` uses the simpler naive base without those constraints. The `reflexion` strategy utilises a two-turn loop in which the model critiques its own initial output for missing rules, imprecise or incorrect numeric conditions, wrong sensor and station names, and duplicate or poorly-structured entries. The

`reflexion_guided` paradigm extends this to three turns, adding a rigid domain-specific checklist in the final turn that covers six categories: (1) completeness across all stations, (2) cross-station dependencies, (3) temporal anomaly patterns (SPIKE, DRIFT, STUCK, OUT_OF_RANGE, CORRELATED), (4) predictive and corrective maintenance protocols (MAINT-01 to MAINT-08), (5) access-control and zone-occupancy rules, and (6) correctness of all numeric threshold values.

- **Level 3 (Tool Use / ReAct):** The `react_abox` paradigm allows the model to actively verify sensor names against a localised Knowledge Graph (ABox). The LLM can halt its generation to emit a `VERIFY: <sensor_name>` command. The system evaluates this request using fuzzy token-overlap matching and returns either a `SENSOR_OK` confirmation or a `SENSOR_NOT_FOUND` warning with up to three syntactically similar suggestions. The loop is bounded to a maximum of five turns; if this limit is reached without a final JSON response, the last assistant message is parsed directly.

7.4 Ensemble Voting and Content Agreement

The `self_consistency` paradigm (Level 2) relies on an ensemble voting mechanism to aggregate the results of three independent LLM calls. To determine whether two independently generated rules describe the same operational constraint, a `_content_agreement` algorithm calculates a weighted similarity score (from 0.0 to 1.0).

Categorical fields (`class`, `station`, `sensor`, `sensorType`, `severity`, `unit`) are evaluated using exact string matching after normalisation, while text fields (conditions and actions) are evaluated using a Jaccard token-overlap approximation. Numeric fields within `ThresholdRule` instances are subjected to a tolerance-based similarity calculation to prevent minor rounding differences from invalidating a match. Field weights are non-uniform: among categorical fields, location identifiers (`station`, `sensor`) carry weight 2.0, rule class (`class`) carries weight 1.5, and the remaining categorical fields (`sensorType`, `severity`, `unit`) each carry weight 1.0. Primary numeric bounds (`critHi`, `critLo`) carry weight 3.0, secondary bounds (`warnHi`, `warnLo`) carry weight 1.5, and text fields carry weight 0.5 for `ThresholdRule` entries (where numerics are the primary signal) or 2.0 for all other rule classes.

The system implements a `score_based` voting strategy by default. Rather than requiring rules to share an identical `ruleId`, this strategy builds a pairwise agreement matrix across all rules generated in the ensemble. If the similarity score meets or exceeds the predefined threshold of 0.55 (i.e. ≥ 0.55), the rules are clustered via greedy merging in descending agreement order. A rule is only retained for the final output if its cluster spans at least 2 out of 3 independent runs. For retained `ThresholdRule` clusters, a secondary majority vote is applied across the threshold-related fields (`critHi`, `critLo`, `warnHi`, `warnLo`) and the measurement unit (`unit`) to resolve minor threshold discrepancies and produce a single stabilised value.

7.5 System Resilience and Output Parsing

Given the tendency of LLMs to deviate from strict schemas, especially under high token loads, the system includes extensive fallback mechanisms. Experiment state is preserved via a JSON registry that marks runs as `partial` upon initiation and `completed` only upon successful disk write (with atomic write-then-

rename saves), allowing interrupted grid searches to resume seamlessly; a per-run partial CSV checkpoint is additionally written after each document. A dedicated heartbeat thread logs a timestamp and the current experiment identifier every 30 seconds, so silent hangs during prolonged API calls are distinguishable from normal long-running inference, and a signal handler persists the registry on interruption so completed runs are never lost.

To extract valid data from malformed LLM responses, a cascading four-strategy JSON parser is deployed. Internal `<think>...</think>` reasoning traces are stripped before any parsing attempt. If standard parsing fails, the system attempts to repair truncated JSON strings by balancing unclosed brackets. If the output contains Markdown artifacts, regex extraction is used to isolate the data blocks. As a final fallback, the parser iterates through the corrupted output to salvage any individually well-formed rule objects.

8. MULTI-AGENT EXTRACTION PIPELINE ARCHITECTURE

The pipeline’s design follows from a single constraint: it is given a directory of operating documents and nothing else. It does not receive a target schema, a rule taxonomy, a list of field names, a worked example, or any indication of whether a document is prose, a table, or a matrix. Every design decision below is a consequence of that constraint, and each is stated with the failure it avoids.

The graph is

$$\text{load_segment} \rightarrow [\text{scout}]^* \rightarrow \text{induce} \rightarrow \text{assemble} \rightarrow [\text{audit}]^* \rightarrow \text{save},$$

where starred stages fan out over the document in parallel and the remainder run once. Figure 1 shows the graph as it is actually constructed, and the rest of this section expands one stage per subsection.

step2_multi_agent_generic.py — LangGraph pipeline
no target schema, no rule taxonomy, no worked example is supplied to any stage

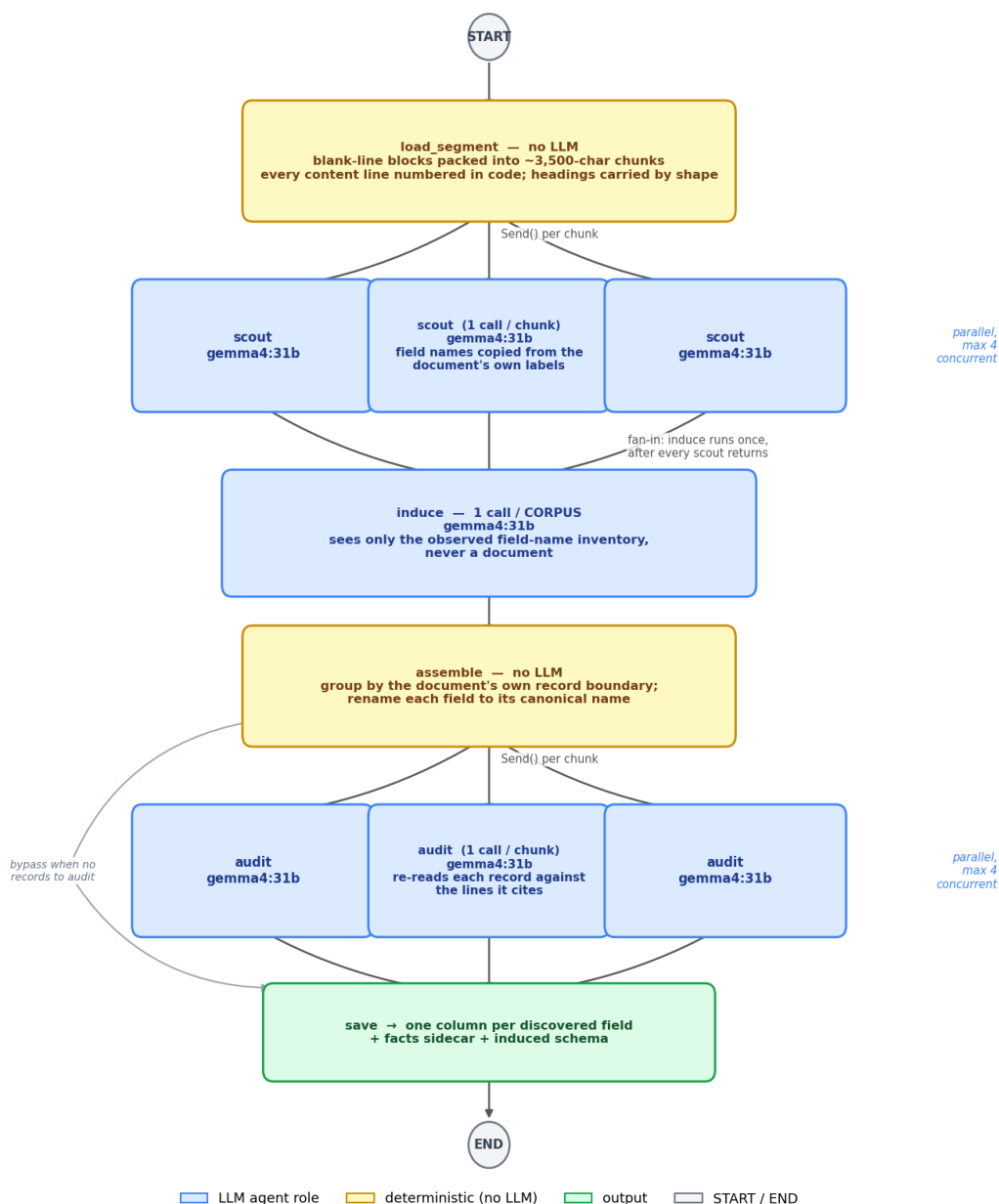


Figure 1: The LangGraph pipeline. Blue nodes are LLM agent roles, yellow nodes are deterministic, green is output. Starred stages in the text correspond to the three-wide rows here: `scout` and `audit` are instantiated once per document chunk and dispatched concurrently, so the three boxes denote “one of these per chunk” rather than exactly three. The grey edge on the left is the conditional bypass taken when no chunk produced records to audit. The figure is generated from the pipeline module’s own constants, so the model names, the chunk size and the concurrency limit shown cannot drift from the code that runs.

Three properties of the topology are worth reading off the figure before the stages are described individually, because each is a design decision rather than an implementation detail.

Two of the six stages use no language model at all. Segmentation and assembly are deterministic code (yellow). This matters for what can be claimed about the output: the boundaries at which text is cut, the line numbers records cite, the grouping of facts into rows, and the renaming of fields to their canonical form are all reproducible operations that do not vary between runs. The non-determinism discussed in Section 6.3.3 enters only at the three blue stages.

The two fan-out stages are per chunk; the inducer is per corpus. Scout and audit scale with document length, so their cost is linear in the corpus and they run at a bounded concurrency. The inducer runs exactly once no matter how large the corpus is, because its input is not the corpus but the inventory of field names the scouts reported. The narrow waist in the middle of the figure is therefore not a drawing convention: it is the one point where the pipeline takes a corpus-wide view.

The scout → induce edge is a fan-in, not a pass-through. Every scout branch must return before the inducer runs, since an arbiter shown one chunk’s vocabulary could not reconcile it with another’s. This is the only synchronisation barrier in the graph, and it is why the inducer cannot be merged into the scout stage however tempting the saving in calls.

8.1 Stage 1: Segmentation and Line Numbering

Segmentation uses no LLM. Each document is split on blank lines into blocks, blocks are packed into chunks of approximately 3 500 characters, and, most consequentially, every content line is assigned a number by code.

Numbering lines in the harness rather than asking the model to track provenance is what makes grounding checkable later. An agent is only ever asked to cite a line number it is currently looking at, which is among the most reliable things a language model can be asked to produce; the alternative, asking a model to reproduce the text it drew from, invites paraphrase and cannot be verified automatically.

Section headings are detected by shape alone (a short, isolated, unpunctuated line) and carried into every chunk they cover. Without this, the answer to “which asset does this table belong to” is lost at the first chunk boundary, because the heading naming the asset sits in the preceding chunk. Detection by shape rather than by a pattern of known heading formats is again a consequence of the no-schema constraint.

8.2 Stage 2: Scout Agents

One LLM call per chunk, issued in parallel with a concurrency of four. The scout reads a numbered chunk and emits one record per rule *the document itself delimits*: a printed rule identifier where the document prints one, otherwise one table row, one matrix cell, or one rule-stating sentence.

Two properties of the scout prompt carry the schema-agnosticism:

- **Field names are copied from the document’s own labels.** A table’s column headers become the record’s field names; a matrix’s axis labels become its subject and field name; a sentence contributes the quantity it names. The scout is never given a set of fields to fill, so it cannot force a document into

a foreign shape, and a corpus whose rules genuinely carry fourteen distinct slots yields fourteen fields because fourteen were observed.

- **No worked example is supplied.** An example demonstrates one particular document shape, and a model shown one reproduces that shape on documents that do not have it. This is the mechanism by which a schema-specified prompt achieves high accuracy on the corpus it was written for and transfers poorly; omitting the example gives up the former to avoid the latter.

The prompt ends with a chain-of-thought instruction: reason through the chunk, then emit JSON. This is the `cot_basic` paradigm, and its selection from the grid of Section 7 is constrained before it is empirical. Only three grid paradigms outscore it, and two of them cannot be carried into a schema-agnostic scout at all: `reflexion_guided`’s final turn is a corpus-specific checklist, and `reflexion` inherits mandatory sensor-naming constraints from its `graph_informed` base, so in both cases the margin is bought with exactly the corpus knowledge this pipeline is defined not to have. The third, `pre_act`, does transfer and is statistically tied with `cot_basic` (0.894 ± 0.009 against 0.892 ± 0.005); `cot_basic` is preferred for its lower run-to-run variance, and because `pre_act`’s preliminary survey of document structure has little to survey on the single chunk a scout reads.

8.3 Stage 3: Schema Induction

One LLM call per corpus, and the only stage with a global view. The inducer never sees a document. Its input is the inventory of field names and category labels the scouts actually produced, with occurrence counts and sample values, and its task is to merge synonyms into one corpus-level schema.

Deriving the schema from what the corpus was observed to contain, rather than proposing one in advance from a document’s opening pages, is what allows the field count to follow the evidence. It also resolves the problem that two documents in one corpus may name the same role differently: reconciliation happens here, once, so that the pipeline’s output is expressed in a single vocabulary even though its scouts spoke several.

Whether that reconciliation improves agreement with a given ground truth is a separate question from whether it is desirable, and the two can diverge: an annotator’s field names are the scoring target, so renaming a field to a canonical form that differs from the annotator’s reduces measured agreement even where the canonical form is the better output. This stage is therefore included on output-quality grounds, and no claim is made here about its effect on $F1_{\text{content}}$.

Withholding the documents from this stage is deliberate. An arbiter that could read the source text would be able to introduce fields no scout reported, which would make the schema a product of the arbiter’s priors rather than of the corpus. Restricting it to the observed inventory means the schema cannot contain anything the corpus did not exhibit.

8.4 Stage 4: Deterministic Assembly

No LLM. Records are grouped by the document’s own record boundary, each field is renamed to its canonical name from the induced schema, and one row per rule is emitted. Because the boundary is the printed

identifier or the physical line the record was read from, granularity is decided by evidence printed in the document rather than by any convention the pipeline carries.

This is the stage that gives rise to the granularity mismatch analysed in Section 6.5.8. Extraction proceeds at the finest granularity the document supports (one atomic fact per cell or value) because that choice loses no information, whereas a coarser choice cannot be undone. Whether a given ground truth annotates at that same granularity is outside the pipeline’s knowledge, and the metric charges the difference as error.

8.5 Stage 5: Grounding Audit

One LLM call per chunk, again in parallel. The auditor re-reads each assembled record against the numbered lines that record cites, and drops or corrects anything not actually stated there. This stage targets numeric hallucination: a threshold the model invented is not present on the line it claims to come from, and the citation created in Stage 1 is what makes that discrepancy checkable in principle.

One safeguard applies. A whole-chunk rejection (an auditor claiming that every record from a chunk is ungrounded) is treated as an audit failure rather than as a finding whenever the chunk carried three or more records, and the records are retained. An auditor asserting that nothing in a sizeable chunk is supported is more likely to be malfunctioning, or to have misunderstood its instructions, than to be correct; without this guard a single bad audit response can silently delete a document’s entire contribution. For a chunk carrying only one or two records the guard does not apply and the rejection stands, because at that size rejecting everything is indistinguishable from the auditor doing its job on a record or two, and no inference of malfunction is available.

The foregoing states the stage’s purpose and mechanism. It is not a claim that the stage raises $F1_{\text{content}}$: what the audit targets is whether a record asserts something its cited line does not support, and Section 10.2 establishes that the metric used here checks that property only weakly. Its contribution is therefore not quantified by the figures reported in this thesis.

8.6 Model Assignment

All three agent roles are served by gemma4:31b, for the reason given in Section 6.3: over 100 grid runs it outperformed the alternative on all ten paradigms, by a margin comparable to the spread between paradigms, and there is no measured basis for assigning any role to the weaker model. Role independence between scout and auditor is available through a command-line override; it is not the default because the only alternative measured here was both weaker and less stable, and an unreliable auditor removes more correct records than incorrect ones.

8.7 Error Resilience

Three failure modes are handled explicitly, each because it was observed to occur against a hosted endpoint rather than anticipated in the abstract.

Empty content from a reasoning model. A model that returns its deliberation in a separate field and its answer in the content field can exhaust the output budget before the answer begins, yielding an empty

content string with a length-based finish reason. Parsing that as an empty result would silently record “this chunk contained no rules”. It is instead raised and retried.

Truncation. A reply cut off at the output budget loses every record after the cut. This is detected from the finish reason and surfaced as a warning rather than absorbed, since the symptom otherwise appears as an unexplained drop in record count.

Transient endpoint failures. Calls are attempted up to three times with exponential backoff from a 15 s base. The timeout is deliberately generous at 600 s: reasoning models with a large output budget legitimately take minutes, and a shorter timeout discards valid work in progress.

9. RESULTS

Every figure in this section is a mean over five repeats of the same configuration, reported with its standard deviation, for the reason given in Section 6.3.3: a single run against a hosted endpoint is one draw, and two configurations whose intervals overlap are tied rather than ranked. Precision is reported alongside the ceiling that Equation 4 imposes on it, since the two are not comparable across corpora without it.

9.1 Extraction Across Four Corpora

Table 7 reports the pipeline on the development corpus and on the three external corpora it had never seen. No prompt, parameter, field mapping, or threshold was changed between the four; the pipeline was invoked identically in each case, differing only in the input directory.

Table 7: Multi-agent pipeline, five repeats per corpus. Rows are ordered as in Table 2: the development corpus first, then the three external corpora in the order they were introduced. The ordering is deliberately independent of the scores, so that the table cannot be read as leading with its best result. *Max P* is the ceiling that the record count imposes on precision (Equation 4); precision must be read against it rather than against 1.0.

Corpus	Role	GT	Records	$F1_{\text{content}}$	P	Max P	R
Production line (iMAKS)	dev	86	116.6 ± 19.7	0.756 ± 0.077	0.668	0.756	0.884
Biogas digestion	external	47	36.4 ± 1.1	0.806 ± 0.022	0.924	1.000	0.715
Cleanroom access	external	37	38.0 ± 0.0	0.747 ± 0.000	0.737	0.974	0.757
RO desalination	external	26	27.2 ± 0.4	0.820 ± 0.045	0.802	0.956	0.839

Four observations follow. The first fixes the reference point, the second is the central result, and the last two constrain how the table should be read.

The development corpus, where the pipeline was built, scores 0.756 ± 0.077 . This is the figure every other number in the table should be compared against, because it is the only corpus whose documents were available while the pipeline was being written. It is reported first for that reason rather than because of where it falls in the ranking. Its recall of 0.884 is the highest of the four and its precision of 0.668 the lowest, a pattern the second and third observations explain.

Performance does not degrade on the three unseen corpora. They score 0.747, 0.806 and 0.820, against the development corpus’s 0.756: two above it, one 0.009 below and well within its spread of 0.077. This is the central result. It is what one expects from a system that received no corpus-specific configuration, and it is not what one would expect from a system whose apparent quality came from having been shaped around its evaluation data. The claim being made is deliberately modest: not that the pipeline performs *better* on unseen material, which the intervals do not support, but that it does not fall away, which is the failure mode a development corpus is meant to expose.

The development corpus is precision-limited by granularity, not by error, and one observation demonstrates this outright. Across the five dev repeats the number of true positives is *identical*: 76 of 86 ground-truth rows matched, every run, giving a recall of 0.884 with zero variance. What differs is verbosity: two runs emitted 95 records and three emitted 131. That alone moves the precision ceiling between 0.905 and 0.656 and the reported F1 between 0.840 and 0.700. The same rules were found each time; only how finely they were itemised changed, and $F1_{\text{content}}$ charged the difference as error. Inspection of the unpaired records confirms that many state facts the document contains and the ground truth simply does not itemise, such as inter-station dependencies recorded in a table that the annotation schema has no row for.

Variance differs sharply across corpora and is not always small. The development corpus’s standard deviation of 0.077 (range 0.700–0.840) is the largest of the four, for the granularity reason just given. Cleanroom by contrast returned an identical 0.747 on all five repeats. A zero standard deviation should not be read as determinism, and a batch of five repeats should not be read as capturing the full variation. An earlier complete batch of five runs per corpus returned different means on *all four* corpora: 0.706 on the development corpus, 0.828 on biogas, 0.720 on cleanroom (again five identical scores), and 0.844 on desalination, against the 0.756, 0.806, 0.747 and 0.820 of Table 7. The log of that batch is retained in the accompanying repository (`logs/final_run.log`); its per-run files were overwritten by the batch reported here. The between-batch movement (up to 0.050, in both directions, with no corpus spared) exceeds some of the within-batch standard deviations, which is an argument for reading every figure in Table 7 as a draw from a wider distribution than five repeats reveal, and it is why two configurations whose intervals overlap are treated as tied throughout this thesis.

9.2 Field-Level Verification Against the Source Documents

$F1_{\text{content}}$ deliberately ignores field identity (Section 6.5.3), so it cannot answer the question a downstream consumer actually needs answered: is each value in the right slot? A knowledge graph populated from these records needs the critical low bound to be the critical low bound, not merely a number that appears somewhere in the row. This section answers that question directly, outside the metric, by comparing cells against the ground truth on the one corpus where a field correspondence can be established without inventing one.

The correspondence is obtained by normalising both sides’ column names (case folding and separator removal) which maps the ground truth’s `critLo`, `warnLo`, `warnHi`, `critHi` onto the pipeline’s `crit_lo`, `warn_lo`, `warn_hi`, `crit_hi`. No vocabulary is consulted; the match is a string operation. Records are keyed by station and sensor, and every one of the ground truth’s 22 threshold sensors is recovered in every

run.

Result: 420/440 bound cells (95.5%) sit in the field the ground truth assigns them to, identically in all five runs. The remaining 20 are the same four cells repeated across runs, and they are worth examining, because they are not extraction errors.

The residual disagreements are ground-truth errors. The source table for ST03_LABELLING reads, under the printed header `Sensor | Unit | CRIT_LO | WARN_LO | Nominal | WARN_HI | CRIT_HI:`

```
| SPD | m/s | 0.65 | 0.75 | 0.85 +/- 0.01 | 0.95 | 1.05 | ...
```

The pipeline extracts `crit_lo = 0.65`, `warn_lo = 0.75`, `warn_hi = 0.95`, `crit_hi = 1.05`, the document’s values, in the document’s columns. The ground truth records 1.05, 1.15, 1.35 and 1.05 for the same four fields. Those numbers appear nowhere in the source, and the annotation assigns `critLo` and `critHi` the same value (1.05), which is not a well-formed threshold range. The same pattern holds for ST04_PACKAGING, where the source states `CRIT_LO = 0.85` and the annotation records 1.25. Scored against the source documents rather than the annotation, field placement on these cells is 440/440.

Two conclusions follow, and both bear on how the headline figures should be read. First, the extraction places values correctly at a rate the row-level metric cannot express: a corpus-level $F1_{\text{content}}$ of 0.756 coexists with essentially exact field placement on the rules that carry numeric bounds. The two measure different things, and the lower number is not evidence of misplaced values. Second, the ground truth against which every figure in this thesis is computed contains errors of its own. This is not a hypothetical concern raised to soften a result: it is four specific cells, in a document any reader can open, and it means reported precision is a lower bound on correctness rather than a measurement of it (Section 10.5).

One practical caveat for any consumer of these records. The column holding the station identifier is not stable across runs: in one run it is `station`, in the others the value sits in `asset` while an empty `station` column also exists. Resolution must therefore take the first non-empty value among candidate columns *per row*, not choose one column per file. This is the naming drift discussed in Section 8.3, appearing exactly where the source document does not print an explicit header for the concept.

9.3 The Cost of Requiring No Schema

The grid of Section 7 supplies the prompt with the development corpus’s schema, rule taxonomy, identifier convention, and worked examples drawn from that same benchmark family. Table 8 reports the leading grid conditions rescored with the metric of Section 6.5, so that both sides of the comparison are produced by the same evaluator on the same ground truth. A figure produced by a different metric could not be placed beside these.

Table 8: Schema-specified single-prompt extraction versus the schema-agnostic pipeline, both scored by $F1_{\text{content}}$ on the development corpus, five repeats each. The grid conditions cannot be run on the three external corpora without their prompts being rewritten by hand.

System	Records	$F1_{\text{content}}$	P	R
gemma4:31b, reflexion_guided (schema given)	78.8	0.946 ± 0.017	0.990	0.907
gemma4:31b, reflexion (schema given)	77.2	0.939 ± 0.008	0.993	0.891
gemma4:31b, cot_basic (schema given)	70.4	0.892 ± 0.005	0.992	0.812
Hand-written regex parser	54.0	0.771	1.000	0.628
Multi-agent pipeline (no schema)	116.6	0.756 ± 0.077	0.668	0.884

Read naively, the table says the schema-agnostic pipeline is beaten by a single prompt and by a regex parser. That reading is wrong in a specific and measurable way, and the correct one is more informative.

Recall is close to equal. The pipeline recovers 0.884 of the ground truth against the best schema-specified condition’s 0.907, 76 of 86 rows against roughly 78. The two systems find nearly the same rules, and the residual gap is an order of magnitude smaller than the F1 gap. Whatever the pipeline mainly lacks, it is not coverage of the ground truth.

The entire gap is precision, and the precision gap is granularity. The grid conditions were told what a row looks like, so they emit 70–79 rows against a ground truth of 86 and enjoy a precision ceiling of 1.0. The pipeline was not told, extracts at the finest granularity the documents support, emits 95–131 records depending on the run, and is capped between 0.905 and 0.656 before any content is examined. The comparison is therefore between a system that was given the annotator’s granularity convention and one that had to guess it.

The schema-specified condition is not a transferable baseline. Its prompt hardcodes field names, class labels, an identifier format, and worked examples containing real station identifiers from the development corpus. Applying it to the biogas, cleanroom, or desalination corpora requires a human to read those documents and rewrite the prompt, which is the per-domain configuration this work exists to eliminate, and, more sharply, is an instance of adapting the extractor to the corpus it is scored on.

The honest summary is therefore this: on a corpus for which somebody has already written a schema, writing that schema into the prompt is worth roughly 0.19 F1. On a corpus for which nobody has, the schema-specified approach has no score at all, and the pipeline reports 0.747–0.820.

10. LIMITATIONS

The limitations below are ordered by how much they bound the claims made in Section 9, most consequential first. Several are properties of the evaluation rather than of the pipeline, and are stated here because a reader cannot otherwise tell how far the reported figures reach.

10.1 Granularity Disagreement Bounds Every Reported Figure

The four corpora disagree about what constitutes one rule, and no convention among them is privileged. Where a ground truth annotates more coarsely than the pipeline extracts, correct records become false positives by the one-to-one assignment (Equation 4); where it annotates more finely (as the development corpus does for threshold rules, splitting one printed table row into up to three annotated rows) ground-truth rows go unmatched and become false negatives instead. Both occur in the same corpus simultaneously (Section 6.5.8), so the discrepancy cannot be removed by a single merge or split rule. On the development corpus the cap moves with the record count (between 0.905 and 0.656 across the five repeats) and the observed precision of 0.668 sits near its lower end.

The consequence is that $F1_{\text{content}}$ as reported here conflates two distinct things: whether the pipeline found the right rules, and whether it itemised them the way a particular annotator did. Recall separates them partially (it is unaffected by surplus records) which is why the near-equal recall between the pipeline and the schema-specified baseline (0.884 against 0.907, Section 9.3) is the more informative comparison. A metric that scored a set of atomic facts against a set of composite rows without requiring a bijection would measure the first thing alone; constructing one is left to future work.

10.2 The Metric Does Not Read Word Order or Attribute Binding

Section 6.6 establishes what $F1_{\text{content}}$ measures, and the same study establishes what it does not. Scrambling token order within a record, against an order-preserving control, costs 0.000; the comparison behaves as a bag of tokens and numbers rather than as a reading of a sentence. Moving quantities between records (the right rule carrying the wrong bound) costs -0.007 , and permuting the category label costs 0.009, so neither value-to-rule binding nor label correctness is checked at all.

Two consequences follow. A record that contains the right words and numbers in a nonsensical arrangement is scored almost as highly as a coherent one. And category-label correctness is essentially unpriced, which is consistent with the design argument that category exact-match is unwinnable across vocabularies, but means the pipeline’s category discovery is not evaluated by the headline figure at all.

10.3 The Metric Has a Non-Zero Floor

Predictions scored against a foreign corpus’s ground truth do not fall to zero. Pooled per corpus the null score is 0.019–0.113, but the worst pairing reaches 0.274, cleanroom predictions against the development corpus’s ground truth, where a large and varied annotation offers the assignment many rows to choose from. Reported figures should therefore be read against a floor of roughly 0.27 for a large ground truth rather than against 0, and a system scoring below that floor would not be distinguishable from one extracting the wrong document entirely.

10.4 One Corpus Is Threshold-Unstable

The acceptance threshold $\tau = 0.6$ is a strict local maximum on none of the four corpora (the curve declines through it on two and is flat through it on two) so it cannot have been chosen to flatter the system. But the

cleanroom corpus moves 0.213 in F1 across a single 0.05 step near that point, where the other three move between 0.000 and 0.052. Its reported figure is therefore contingent on the threshold in a way the others are not, and should be quoted with its curve. Relatedly, the ranking of corpora by $F1_{\text{content}}$ is not stable across the threshold sweep, so no claim that one corpus is intrinsically harder than another is supported.

10.5 Ground-Truth Incompleteness

Two distinct annotation problems affect the reported figures, and they push in the same direction.

Omission. Inspection of the development corpus’s unpaired records shows that a substantial share state facts the source document contains but the annotation schema has no row for: inter-station dependencies recorded in a table the ground truth does not cover, or the `nominal` column of the threshold tables, which the documents print and the annotation drops. These are counted as false positives.

Error. The ground truth is also, in places, simply wrong. Four bound cells for the two SPD sensors record values that appear nowhere in the source documents, and for `ST03_LABELLING` they assign `critLo` and `critHi` the same value, which is not a well-formed range. The pipeline extracts the printed values correctly and is scored as incorrect for doing so (Section 9.2). These are counted as false positives. The reported precision is therefore a lower bound on the fraction of records that are *correct*, and an accurate measure only of the fraction that are *annotated*. Quantifying the difference would require re-annotating the corpora at the pipeline’s granularity, which would compromise their independence.

10.6 Results Are Not Bit-Reproducible

Every agent role is served by a hosted endpoint whose long generations are not reproducible under a fixed seed, for the reasons given in Section 6.3.3. Repeats therefore differ, and every figure is a mean over five runs with its standard deviation. Exact replay is available only through the response cache, which is keyed on the request content and is disabled whenever repeats are requested, so the reported spreads are real, but no single number in this thesis can be reproduced exactly without that cache. The development corpus’s standard deviation of 0.077, spanning 0.700–0.840, shows the spread is not always negligible. Nor do five repeats bound the variation: a complete earlier batch of five runs per corpus returned means differing from the reported ones by up to 0.050, on every corpus (Section 9.1).

10.7 Corpus Scale and Composition

The four corpora comprise seven documents and 196 annotated rules in total, with three of the four external corpora consisting of a single document each. This is enough to show that performance does not collapse on unseen material, and not enough to characterise how performance varies with document length, corpus size, or domain distance. All documents are synthetic and English; real industrial SOPs would introduce annotation noise, inconsistent formatting, and multilingual content that this evaluation does not probe.

10.8 Five Repeats Bound the Statistical Claims

Five runs per configuration supports reporting a mean and a standard deviation and supports declaring two overlapping intervals tied. It does not support significance testing, and no such test is claimed. Where two figures differ by less than the standard deviations separating them, this thesis reports them as indistinguishable rather than ranking them.

10.9 Single Model, Single Endpoint

All three agent roles are served by one model, `gemma4:31b`, selected on grid evidence over one alternative. Whether the architecture’s behaviour generalises across model families is untested, and the specific finding that role independence between scout and auditor is not worth its cost rests on a comparison with a single weaker alternative rather than on a survey.

10.10 No Downstream Validation

This thesis evaluates rule extraction against annotated ground truth. It does not demonstrate that the extracted rules are sufficient to drive a downstream application such as knowledge-graph population or sensor-stream anomaly detection. A high $F1_{\text{content}}$ establishes agreement with an annotator, which is a necessary but not a sufficient condition for operational usefulness; the two can diverge, for instance where the metric’s insensitivity to attribute binding (Section 10.2) admits a record whose bound is attached to the wrong sensor. The accompanying repository contains preliminary downstream tooling of exactly this kind (knowledge-graph population and sensor-stream anomaly detection over extracted rules, under `layers/layer_4`); it is not integrated with the pipeline evaluated here and none of its output is reported in this thesis. Establishing operational sufficiency requires closing that loop and is left to future work.

A. COMPLETE EVALUATION RESULTS

This appendix lists every scored condition. All figures are produced by the same evaluator (Section 6.5) with an acceptance threshold of 0.6, so every row is directly comparable to every other. *Records* is the mean number of extracted rows; *Max P* is the ceiling the record count imposes on precision (Equation 4).

Multi-Agent Pipeline (no schema supplied)

Table 9: Schema-agnostic multi-agent pipeline, five repeats per corpus. Development corpus above the rule, external corpora below it, ordered as in Table 2 rather than by score.

Corpus	GT	Records	F1	SD	P	Max P	R
dev_production_line	86	116.6	0.756	0.077	0.668	0.756	0.884
external_test_biogas	47	36.4	0.806	0.022	0.924	1.000	0.715
external_test_cleanroom	37	38.0	0.747	0.000	0.737	0.974	0.757
external_test_desalination	26	27.2	0.820	0.045	0.802	0.956	0.839

Schema-Specified Grid, Rescored with the Current Metric

All conditions below were prompted with the development corpus’s schema, taxonomy, identifier convention and worked examples (Section 7.1), and are scored against that corpus only. They cannot be run on the external corpora without their prompts being rewritten.

Table 10: All grid conditions and the regex baseline, rescored by $F1_{\text{content}}$ on the development corpus.

Condition	Runs	Records	F1	P	R
gemma4-31b_reflexion_guided	5	78.8	0.946	0.990	0.907
gemma4-31b_reflexion	5	77.2	0.939	0.993	0.891
gemma4-31b_pre_act	5	70.6	0.894	0.992	0.814
gemma4-31b_cot_basic	5	70.4	0.892	0.992	0.812
gemma4-31b_naive	5	66.2	0.859	0.988	0.760
gemma4-31b_cot_structured	5	63.6	0.850	1.000	0.740
gemma4-31b_self_consistency	5	63.0	0.846	1.000	0.733
gemma4-31b_few_shot_static	5	62.2	0.840	1.000	0.723
gemma4-31b_graph_informed	5	62.0	0.835	0.997	0.719
nemotron-3-nano-30b_reflexion_guided	5	67.6	0.833	0.949	0.744
nemotron-3-nano-30b_self_consistency	5	60.4	0.825	1.000	0.702
nemotron-3-nano-30b_cot_basic	5	62.0	0.822	0.983	0.709
nemotron-3-nano-30b_naive	5	60.4	0.819	0.993	0.698
gemma4-31b_react_abox	5	59.2	0.814	1.000	0.688
nemotron-3-nano-30b_reflexion	5	79.0	0.811	0.888	0.774
nemotron-3-nano-30b_few_shot_static	5	67.2	0.786	0.924	0.702
nemotron-3-nano-30b_pre_act	5	97.8	0.783	0.804	0.828
baseline_b0	1	54.0	0.771	1.000	0.628
nemotron-3-nano-30b_graph_informed	5	57.4	0.767	0.961	0.642
nemotron-3-nano-30b_cot_structured	5	78.4	0.729	0.859	0.695
nemotron-3-nano-30b_react_abox	5	49.8	0.707	0.972	0.563

REFERENCES

- [1] [author details required]. [unverified citation — fill in] brex, [year required]. Bibliographic details not verified; supply full reference before submission.
- [2] [author details required]. [unverified citation — fill in] cs230: Deep learning (stanford university course), [year required]. Bibliographic details not verified; supply full reference before submission.
- [3] [author details required]. [unverified citation — fill in] cs520: Knowledge graphs (stanford university course), [year required]. Bibliographic details not verified; supply full reference before submission.
- [4] [author details required]. [unverified citation — fill in] de jure, [year required]. Bibliographic details not verified; supply full reference before submission.
- [5] [author details required]. [unverified citation — fill in] imaks: industrial multi-agent knowledge extraction synthetic benchmark, [year required]. Bibliographic details not verified; supply full reference before submission.
- [6] [author details required]. [unverified citation — fill in] karma: multi-agent llm framework for knowledge graph enrichment, [year required]. Bibliographic details not verified; supply full reference before submission.
- [7] [author details required]. [unverified citation — fill in] kg-pipe, [year required]. Bibliographic details not verified; supply full reference before submission.
- [8] [author details required]. [unverified citation — fill in] knowledge graph construction from technical documentation, [year required]. Bibliographic details not verified; supply full reference before submission.
- [9] [author details required]. [unverified citation — fill in] knowledge graphs for industrial digital twins, [year required]. Bibliographic details not verified; supply full reference before submission.
- [10] [author details required]. [unverified citation — fill in] multi-agent systems for information extraction, [year required]. Bibliographic details not verified; supply full reference before submission.
- [11] [author details required]. [unverified citation — fill in] sop-rag, [year required]. Bibliographic details not verified; supply full reference before submission.
- [12] [author details required]. [unverified citation — fill in] sopbench, [year required]. Bibliographic details not verified; supply full reference before submission.
- [13] [author details required]. [unverified citation — fill in] tabagent, [year required]. Bibliographic details not verified; supply full reference before submission.
- [14] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

- [15] J. Harry Caufield, Harshad Hegde, Vincent Emonet, et al. Structured prompt interrogation and recursive extraction of semantics (SPIRES): A method for populating knowledge bases using zero-shot learning. *Bioinformatics*, 40(3), 2024.
- [16] Mohamad Yaser Jaradeh, Kuldeep Singh, Markus Stocker, and Sören Auer. PLUMBER: A modular framework to create information extraction pipelines. In *Companion Proceedings of the Web Conference (WWW)*, 2021.
- [17] Ziwei Ji, Nayeon Lee, Rita Frieske, et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- [18] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.
- [19] Shirui Pan, Linhao Luo, Yufei Wang, et al. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024.
- [20] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.
- [21] Štěpán Šimsa, Milan Šulc, Michal Uříčář, et al. DocILE benchmark for document information localization and extraction. In *International Conference on Document Analysis and Recognition (ICDAR)*, 2023.
- [22] Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.
- [23] Yuan Yao, Deming Ye, Peng Li, et al. DocRED: A large-scale document-level relation extraction dataset. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 764–777, 2019.